

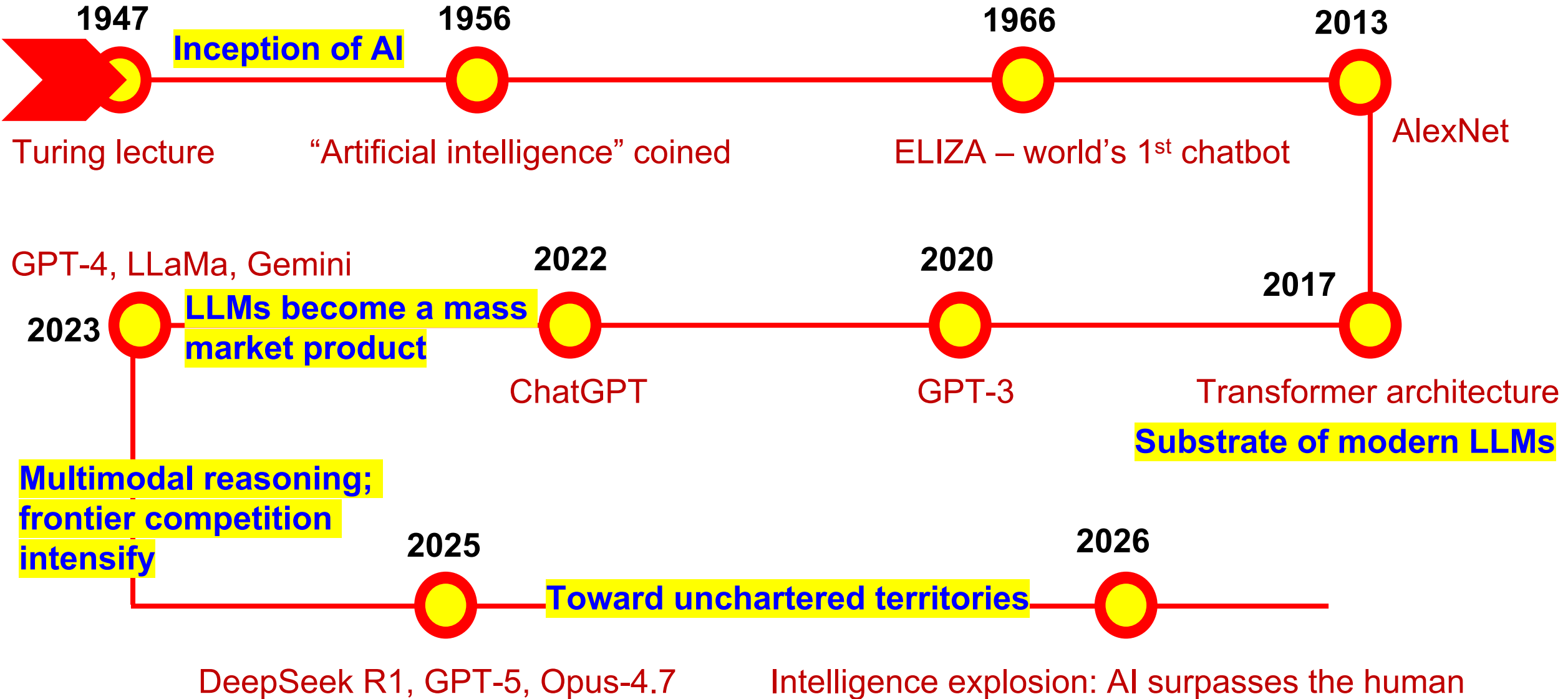
Agentic AI for Physical Design: From Flow Tuning to Self-Evolving Tools

UCSD ECE 260C Spring 2026

Bodhi Pramanik

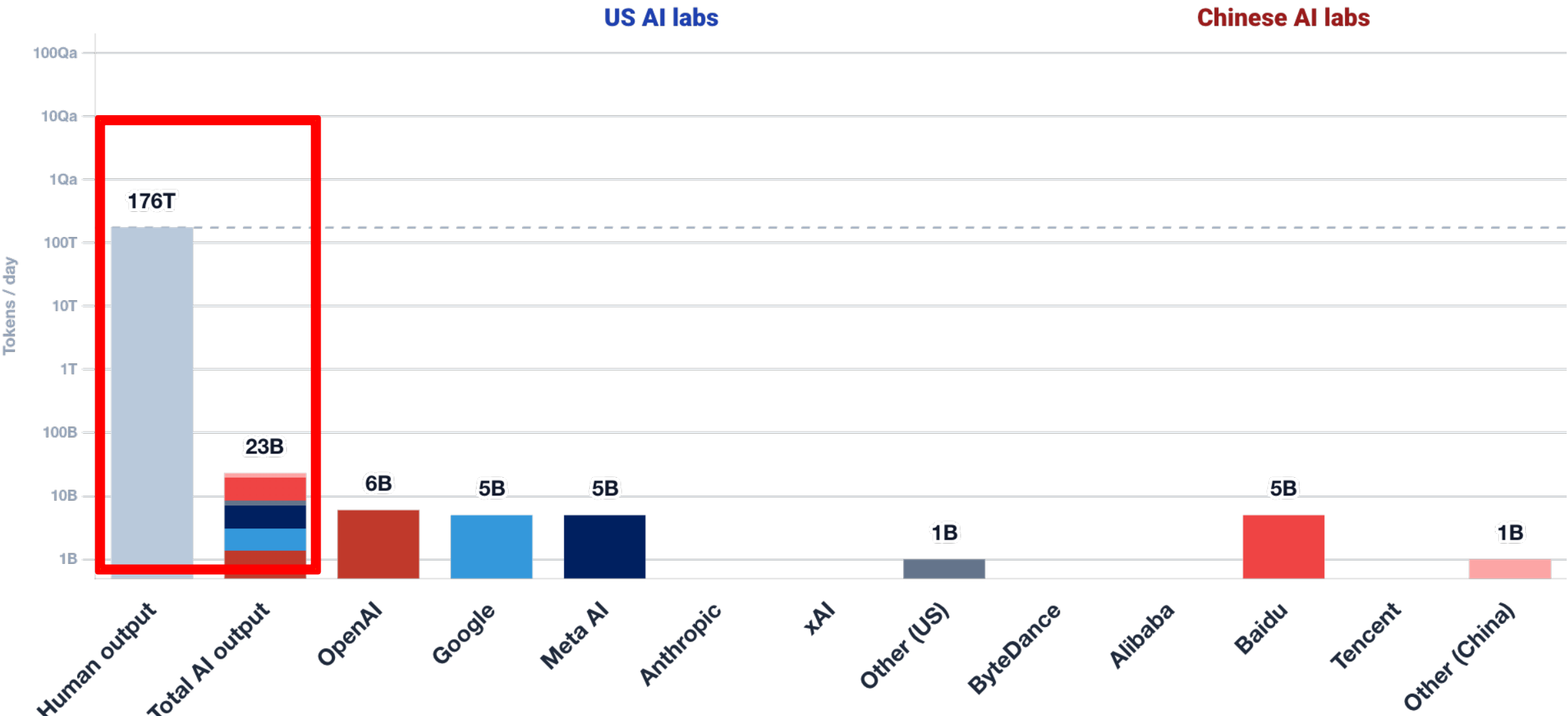
bopramanik@ucsd.edu

AI acceleration through the ages of time



Intelligence explosion is real (2021)

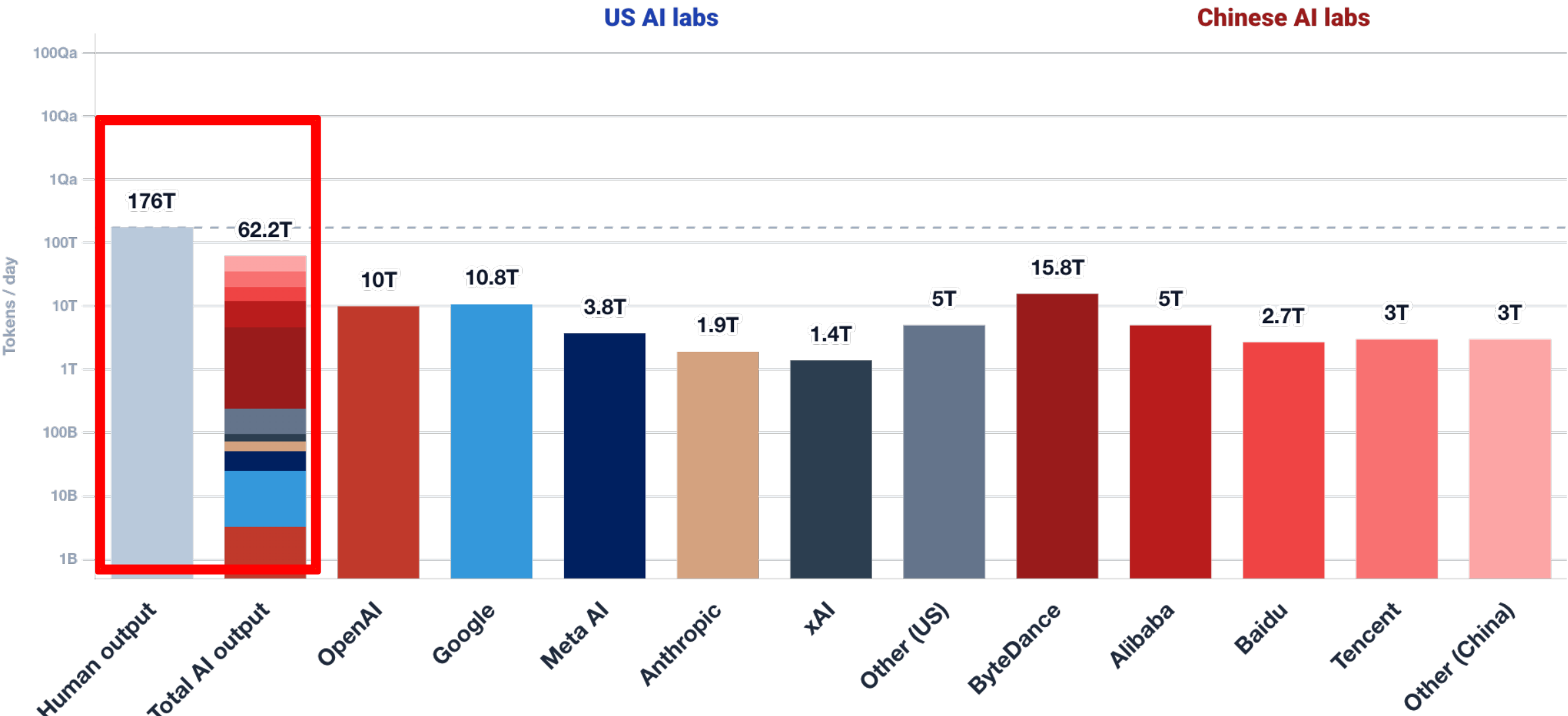
Aggregate daily AI output vs aggregate daily human output



Courtesy: LifeArchitect.ai

Intelligence explosion is real (2025)

Aggregate daily AI output vs aggregate daily human output



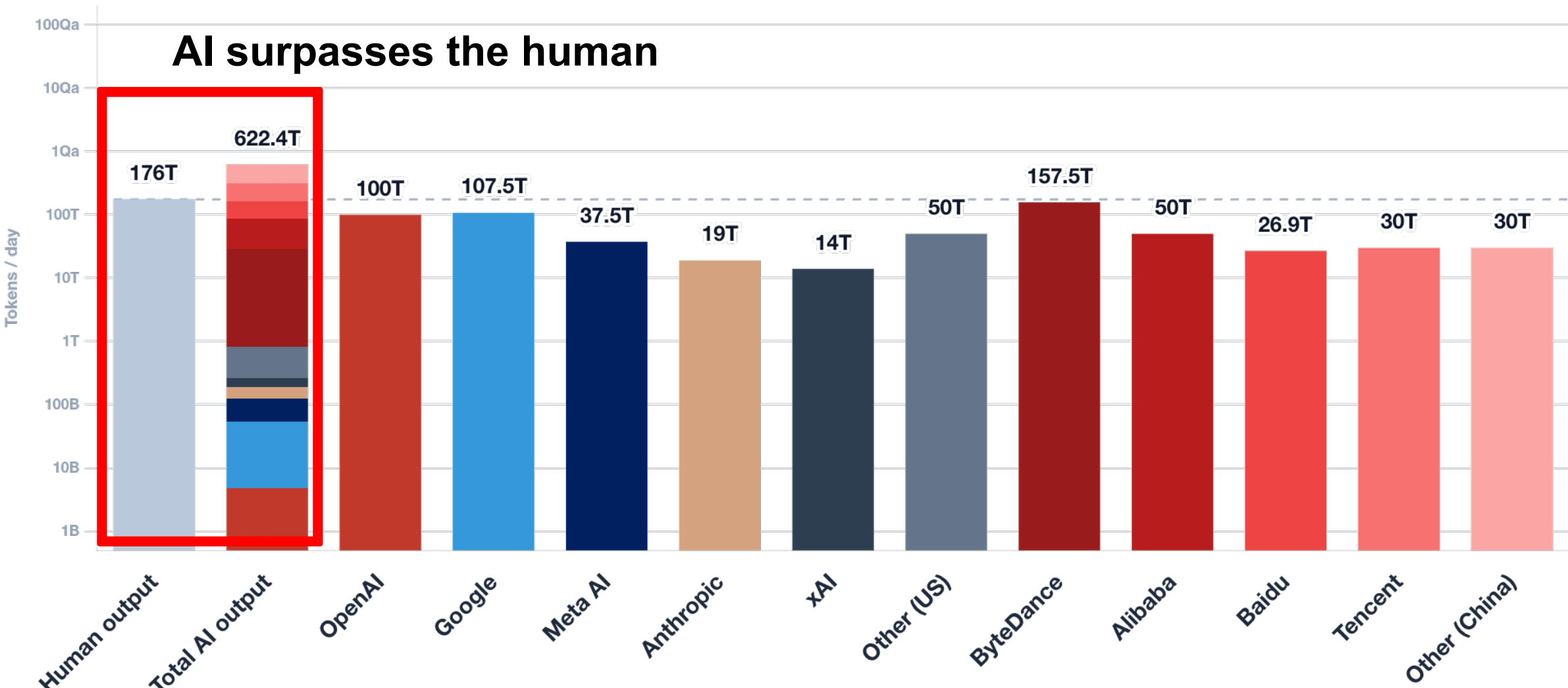
Courtesy: LifeArchitect.ai

Intelligence explosion is real (2026)

Aggregate daily AI output vs aggregate daily human output

US AI labs

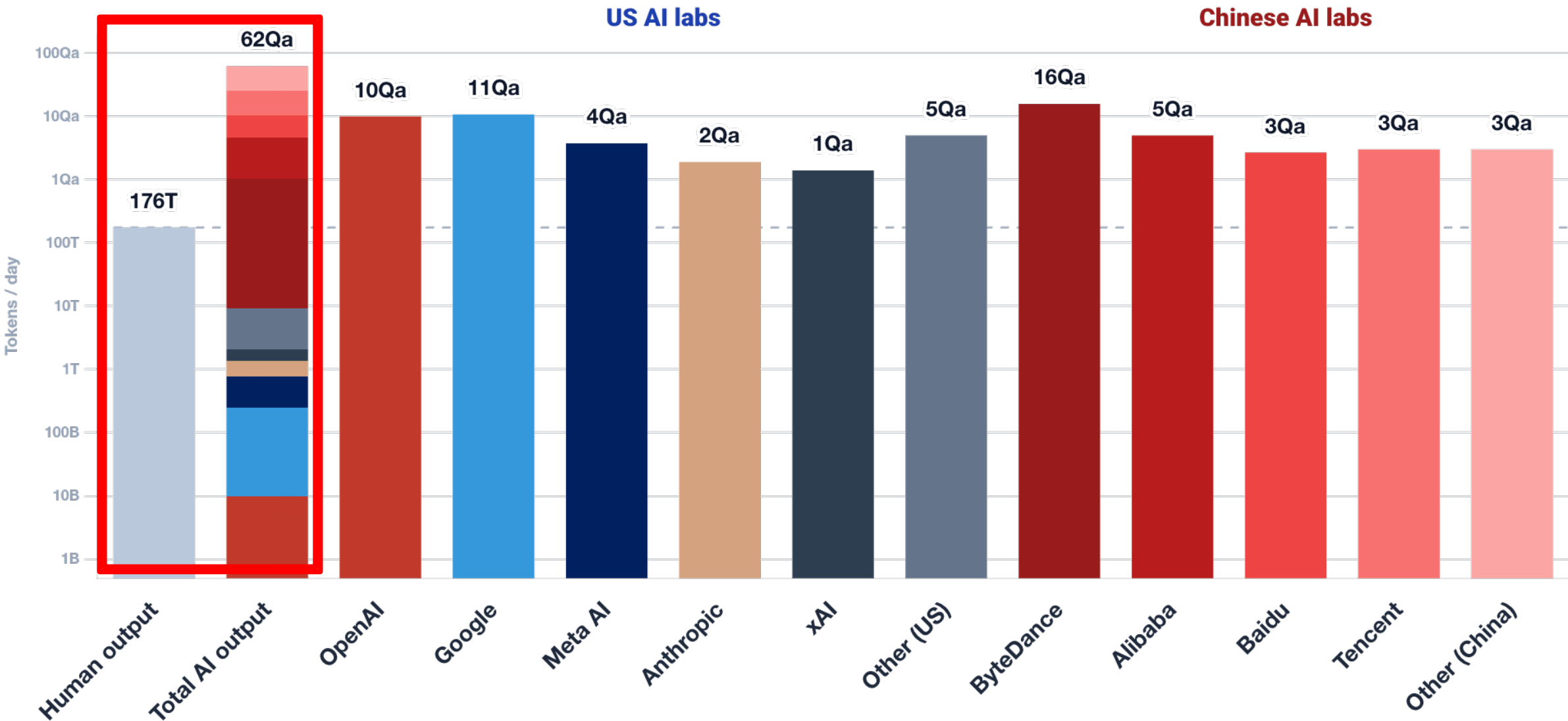
Chinese AI labs



Courtesy: LifeArchitect.ai

Intelligence explosion is real (projection 2028)

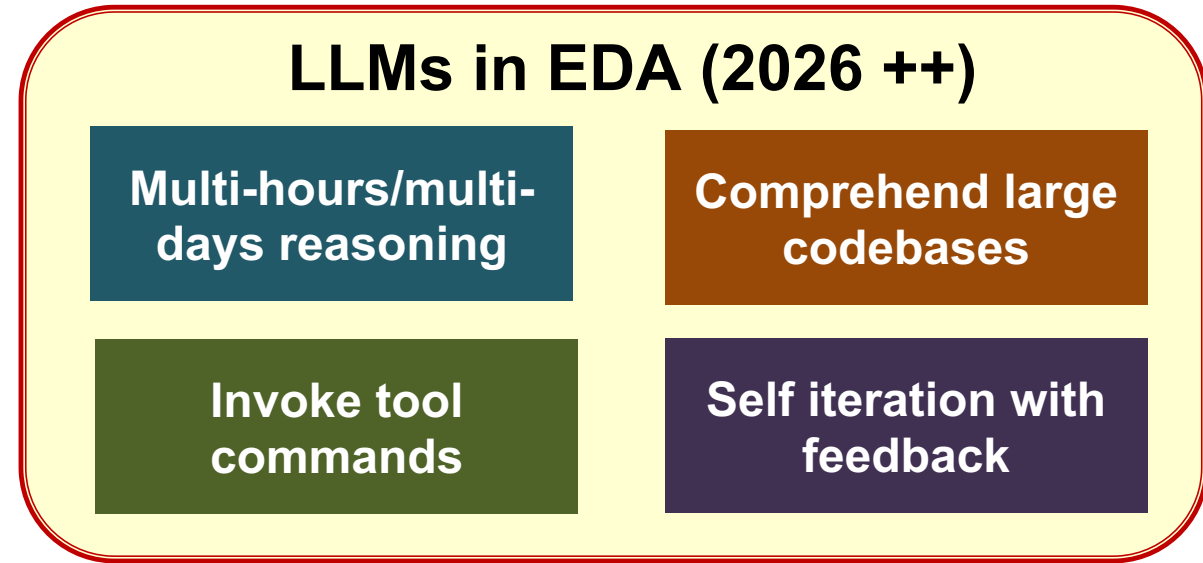
Aggregate daily AI output vs aggregate daily human output



Courtesy: LifeArchitect.ai

The agentic lever (*LLMs → Agents → R&D Acceleration*)

- Scaling is **design-based**
- Tools are **bespoke**
- Ecosystem is **consolidated**
- **Optimization imperative**
= do better and faster, with less



IC design optimization
(PPA, schedule, cost)



LLMs + tool use



EDA = tools
(synthesis, P&R, STA, etc.)



(Human +) Agentic R&D loop
(read logs, tweak params, edit code, check/eval + iterate)

From “EDA **tools**” ➡ to “EDA R&D **collaborators**”

From assistance to autonomy: **two levels**

- **Fact:** Agents (in EDA) will operate across differing levels of **abstraction**
 - **Abstraction** determines **search space**, **feedback signal**, and **risk**
 - *Open-source EDA is the perfect (and our only) testbed !!!*
- **Two levels (~patterns)**
 - **Code-level agents** modify algorithmic structure (source code)
 - ← "Improve the global swap operator in the dpl module of OpenROAD"
 - **Flow-level agents** modify control variables (tool flow, parameters, objectives)
 - ← "Minimize ECP with < 2% area/power/PDP degradation by tuning ORFS configs"
- *Is the agent reasoning about **heuristics** (code and operators) or about **policies** (parameters and sequencing)? (Inevitably, agents will reason about both!)*

Code-level and flow-level agents are **complementary**

- True autonomy in EDA requires reasoning at **both levels**

Flow-level agents

- Choose objectives
- Tune configurations
- Decide which tools / sequences to run
- Optimize system-level PPA

Flow-level agent
(Objectives, configs, orchestration)

Tool flow execution

Code-level agent
(Heuristics, operators, algorithms)

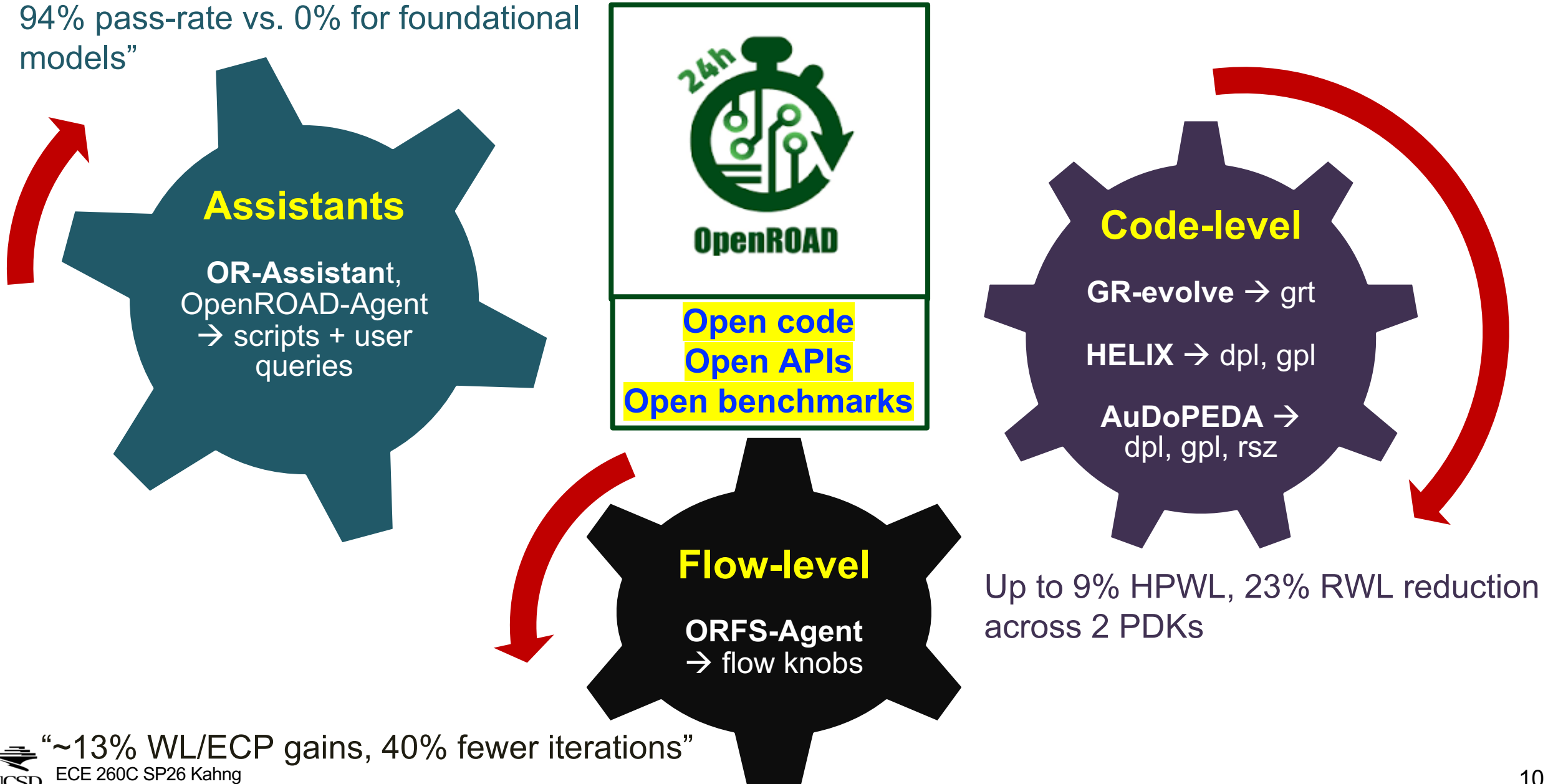
Code-level agents

- Modify algorithms
- Improve heuristics
- Literature grounding
- Refactor and extend tool internals
- Expand the design space itself

- Flow-level agents **optimize within the current design space**
- Code-level agents **reshape the design space itself**

OpenROAD as a foundation for agentic EDA research

94% pass-rate vs. 0% for foundational models”



Three frontiers of recent progress in OpenROAD

Flow tuners

- Treat EDA tools as **black-boxes**; **tune** parameters and orchestrate flows
- **ORFS-Agent** (UCSD)
OpenROAD-Agent (ASU)
ORAssistant (PII)
- **Algorithms** **unchanged**; agents learn the policy

“~40% fewer iterations than autotuners”



Repo-scale coding

- Agents can **read, edit, compile** and **benchmark** across **multi-language codebases**
- **AuDoPEDA** (UCSD)
- **Algorithms** **change**; formal verification; closed-loop guardrails

“~19% power gains”



Bespoke, design-adaptive tools

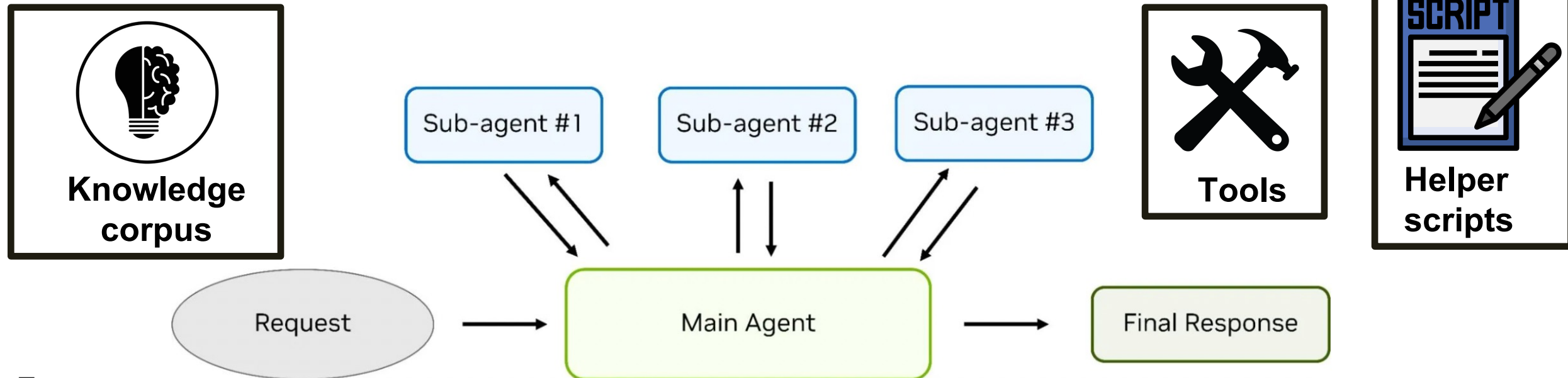
- **Per-design** code evolution; tool **specializes** to the workload
- **HELIX** (UCSD)
GR-Evolve (ASU)
- **Algorithms** **change**; **one tool per design** vs. one tool for all designs

“~9% HPWL gains”



Modern agentic frameworks follow a **standard architecture**

- **Primary agent:** responsible for delivery of entire task end-to-end; may orchestrate sub-agents that tackle subtasks; powered by the strongest model
- **Sub-agents:** spawned by the primary agent to handle narrower tasks; often powered by weaker models
- **File system statefulness:** derived from agents writing memory and tool call output to files ← serves as a method of context management
- **Corpus:** helps ground the LLM for context-relevant reasoning



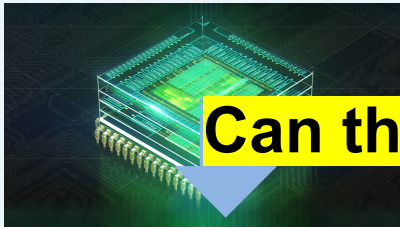
Evolving Global and Detailed Placement

HELIX: Placement **He**uristics Evolution Via
LLM-**I**nstructed Code **Ex**ploration

Challenges of algorithm design

- Domain expertise knowledge
- Trial-and-error
- Intensive manpower

Real-world applications

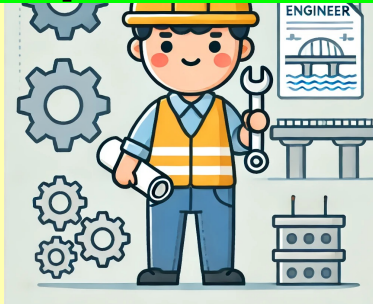


Chip Design

New challenges
Buy optimality

Can this cycle be made “easier” / “better” ?

FunSearch, AlphaEvolve, OpenEvolve, EoH ?



CAD / EDA Engineer



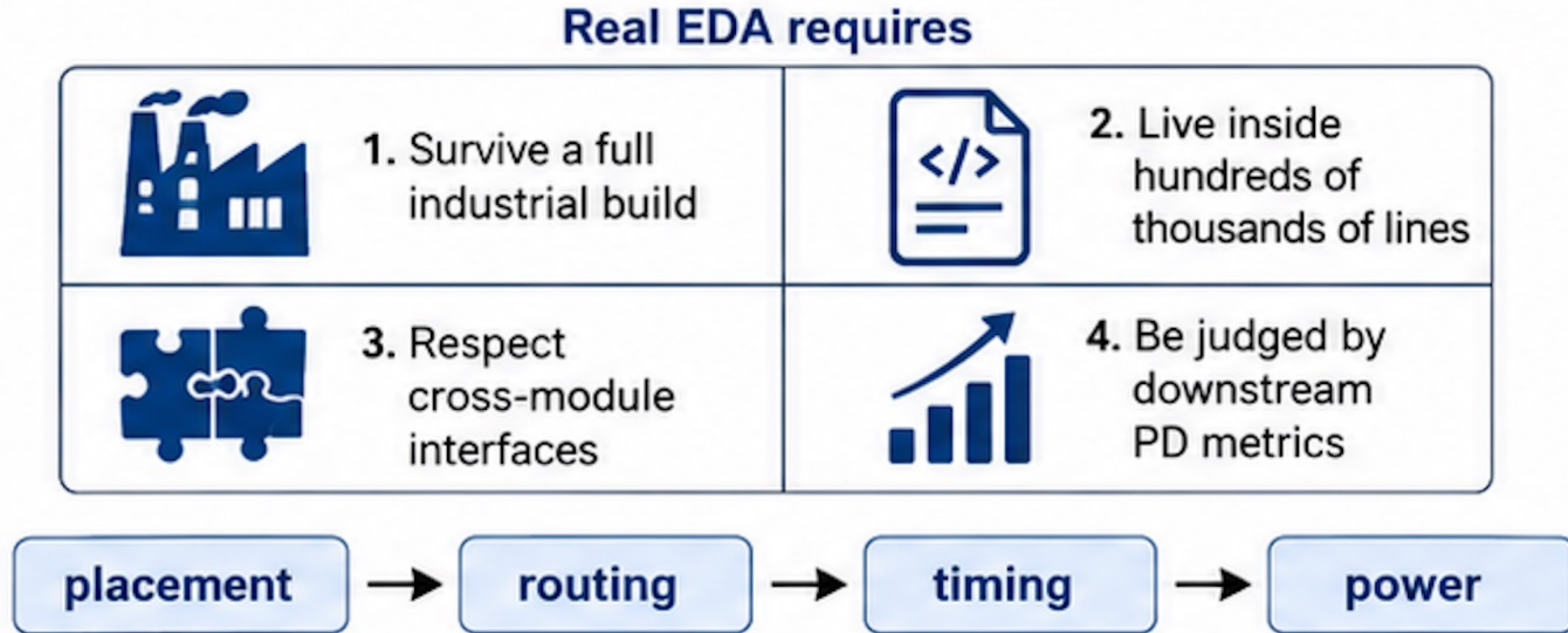
Theorist / Algorithm Designer



Programmer

LLM-guided code evolution for EDA codebase

- Prior LLM + evolution work (FunSearch, AlphaEvolve, AlphaDev, etc.) targets small, self-contained programs
- **Typical assumptions:** minimal interfaces, trivial compilation, cheap evaluation overhead
- **Real EDA breaks all four assumptions**



HELIX – problem formulation

- **Goal: evolve OpenROAD's placement algorithms (gpl, dpl)**

Search space Θ = all compilable C++ implementations
reachable from a baseline via source edits

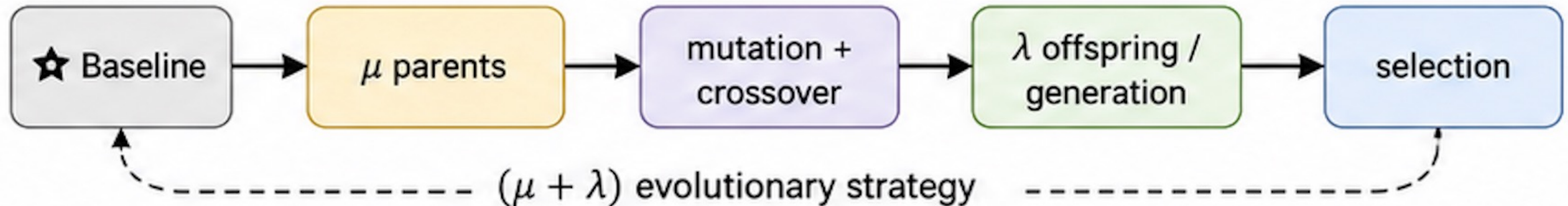
*A discrete, high-dimensional space where most points do not compile,
let alone improve quality.*

Candidate θ : a modified version of the default OpenROAD placer

parameter tuning

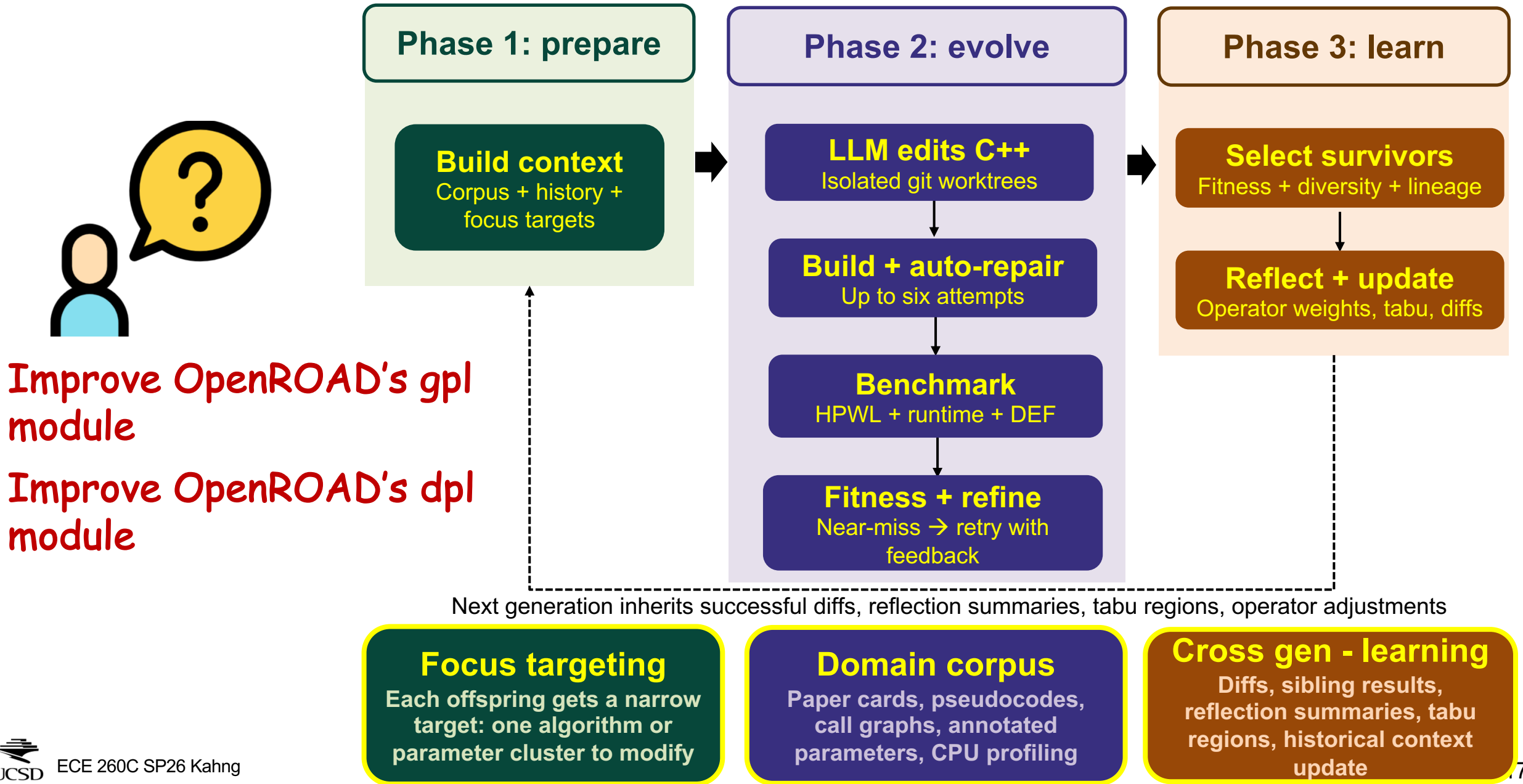
logic restructure

new optimization pass



$$\text{Fitness}(\theta) = \text{reward}(\text{HPWL ratio vs. baseline}) - \text{runtime penalty}(\gamma_g) - \text{failure penalty } \zeta$$

How HELIX works – 3 phase workflow



Knowledge corpus: what the LLM sees

Knowledge layers

> 100 literature cards

Distilled from placement papers → JSON

> 30 pseudocode files (15.6K lines)

Algorithm level – not raw C++

Call graphs (> 1500 functions)

Static analysis via Clang

Parameter catalog (~100)

File:line, type, description

CPU profiles

What functions are the hotspots?

Architecture guide +

LLM

Example: what the LLM sees?

Literature card (RePIAce)

```
"technique": "Nesterov + density penalty"  
"pseudocode": "while overflow > target:  
  grad = WL_grad +  $\lambda$  · density_grad"  
"claim": "3-5% HPWL vs. ePlace"  
"risk": "sensitive to step-length init"
```

Parameter catalog entry

```
name: initialPrevCoordiWeight  
file: nesterovPlace.cpp:342  
range: [0.5, 2.0]  
default: 1.5
```

CPU profiling (DPL, aes design)

```
Random Improvement 38% CPU  
Global Swaps 25% CPU
```

The corpus replaces the domain expert's intuition – without it the LLM defaults to surface-level edits – *parameter scaling + structural tweaks*

Five code mutation operators in HELIX

- **Adaptive scheduling of operators:** safe operators dominate early, then probability mass shifts to lit / creative
- **Focus targeting** steers each offspring at one unique code region

mutate_param

Tune 1 – 3 paraters within catalog

Budget: 25 lines

Small, safe, reliable

mutate_struct

Restructure control flow or data access

Budget: 100 lines

Reorder, add early exits

mutate_lit

Implement ideas / techniques from a paper

Budget: 200 lines

Paper → production C++

mutate_creative

Implement ideas / techniques from a paper

Budget: 200 lines

Paper → production C++

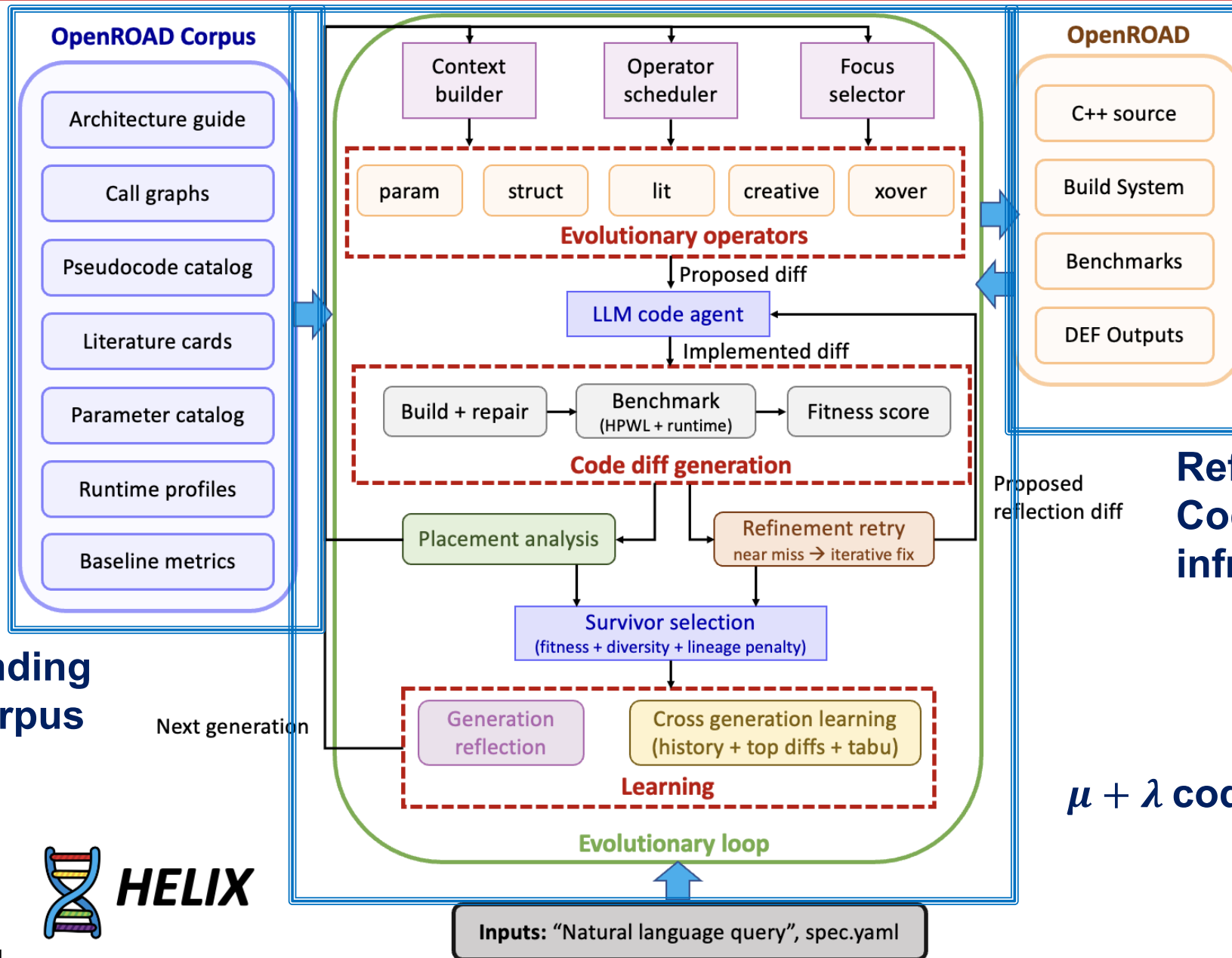
crossover

Merge diffs from two parents

Budget: dynamic

Combine strengths of multiple approaches

Overall HELIX architecture



Domain-grounding
knowledge corpus

Reference codebase;
Code building
infrastructure;

$\mu + \lambda$ code evolution



HELIX evolved placers – excerpt of some diffs

```
// in src/gpl/src/nesterovPlace.cpp — NesterovPlace::init()
for (auto& nb : nbVec_) {
    npUpdateCurGradient(nb);
+   float wlSum = nb->getWireLengthGradSum();
+   float densSum = nb->getDensityGradSum();
+   if (wlSum > 0 && densSum > 0) {
+       nb->setDensityPenalty((wlSum / densSum) *
npVars_.initDensityPenalty);
+   }
nb->update
```

gpl

**Design-adaptive density
penalty init (mutate_struct)**

```
// in src/gpl/src/nesterovPlace.cpp — updateNextIter()
+ // HPWL-velocity triggered WL coefficient ratchet
+ constexpr float kVelocityThresh = 0.005f;
+ constexpr float kRatchetStep = 0.010f;
+ constexpr float kRatchetDecay = 0.97f;
+ constexpr float kMaxRatchet = 0.06f;
+ if (prevHpwl_ > 0 && average_overflow_unscaled_ < 0.25f
+     && average_overflow_unscaled_ > 0.08f) {
+     const float velocity = static_cast<float>(hpwl - prevHpwl_)
+         / static_cast<float>(prevHpwl_);
+     if (velocity > kVelocityThresh) {
+         wl_coef_ratchet_ = std::min(kMaxRatchet,
wl_coef_ratchet_ + kRatchetStep);
+     } else {
+         wl_coef_ratchet_ *= k
```

gpl

**Adaptive gradient restart
(mutate_creative)**

```
// in src/gpl/src/nesterovPlace.cpp — doNesterovPlace()
-   const float coeff = (prevA - 1.0) / curA;
+   float coeff = (prevA - 1.0) / curA;
+   // Overflow-adaptive momentum damping
+   constexpr float kDampingThreshold = 0.15f;
+   constexpr float kMinDamping = 0.65f;
+   if (nesterov_iter > start_iter
+       && average_overflow_unscaled_ < kDampingThreshold) {
+       float ratio = average_overflow_unscaled_ / kDampingThreshold;
+       coeff *= std::max(kMinDamping, std::sqrt(ratio));
+   }
```

gpl

**Dynamic momentum
dampening (mutate_lit)**

```
// in src/dpl/src/optimization/detailed_reorder.cxx — reorder(bool reverse)
+ // Segment processing order: forward passes prioritize denser segments
+ std::vector<int> segOrder(numSeg);
+ ...
+ if (!reverse) {
+     std::vector<int> cellCount(numSeg);
+     ...
+     std::ranges::sort(segOrder, [&cellCount](int a, int b) {
+         if (cellCount[a] != cellCount[b])
+             return cellCount[a] > cellCount[b];
+         return a < b;
+     });
```

dpl

**Density priority segment
reordering (xover)**

Comparisons with default OpenROAD (NG45)

HELIX-evolved gpl and dpl

Design	<i>post-dpl-HPWL</i>		<i>RWL</i>		<i>WNS (ps)</i>		<i>TNS (ns)</i>	
	OR-def	OR-evo	OR-def	OR-evo	OR-def	OR-evo	OR-def	OR-evo
aes	208682	-8 %	258337	-6 %	-64.03	-86.01	-3.75	-4.37
ibex	197361	-1 %	255223	-2 %	-146.02	-137.01	-24.54	-57.03
jpeg	471729	-5 %	561271	-5 %	-105.01	-98.00	-36.62	-35.13
cva6	2040118	-9 %	2409729	-9 %	-114.52	-80.80	-4.90	-4.80
swerv	3638329	-8 %	4169496	-8 %	-136.00	-76.04	-27.53	-15.75
ariane37	3993809	-1 %	4621172	-1 %	-183.01	-125.01	-35.56	-7.81

Evolving Global Routing

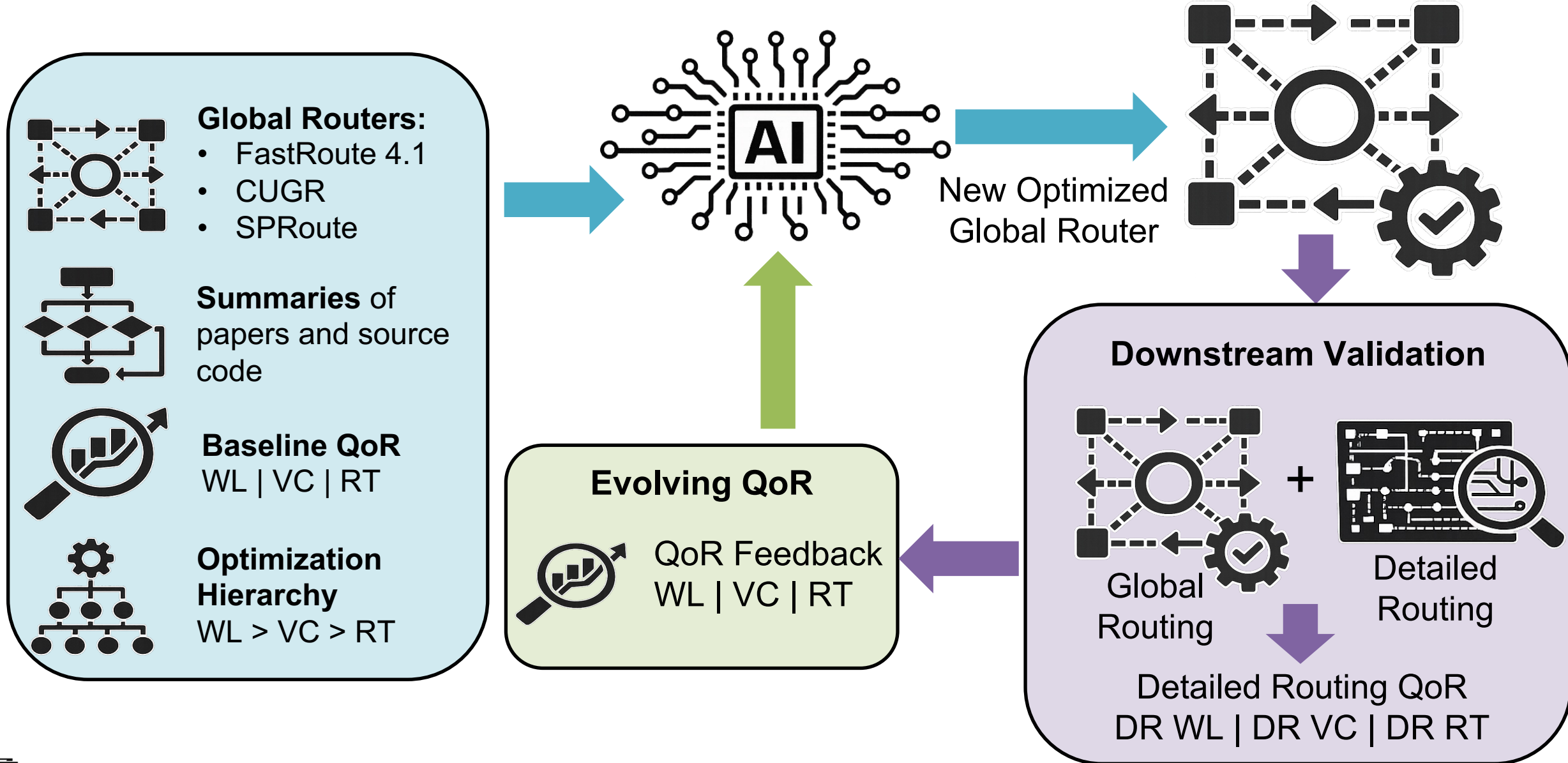
Slides courtesy: Vidya Chhabria (ASU)

GR-Evolve: Design-Adaptive Global Routing via LLM-Driven Algorithm Evolution

Taizun Jafri
Arizona State University
Tempe, Arizona, US
tsjafri@asu.edu

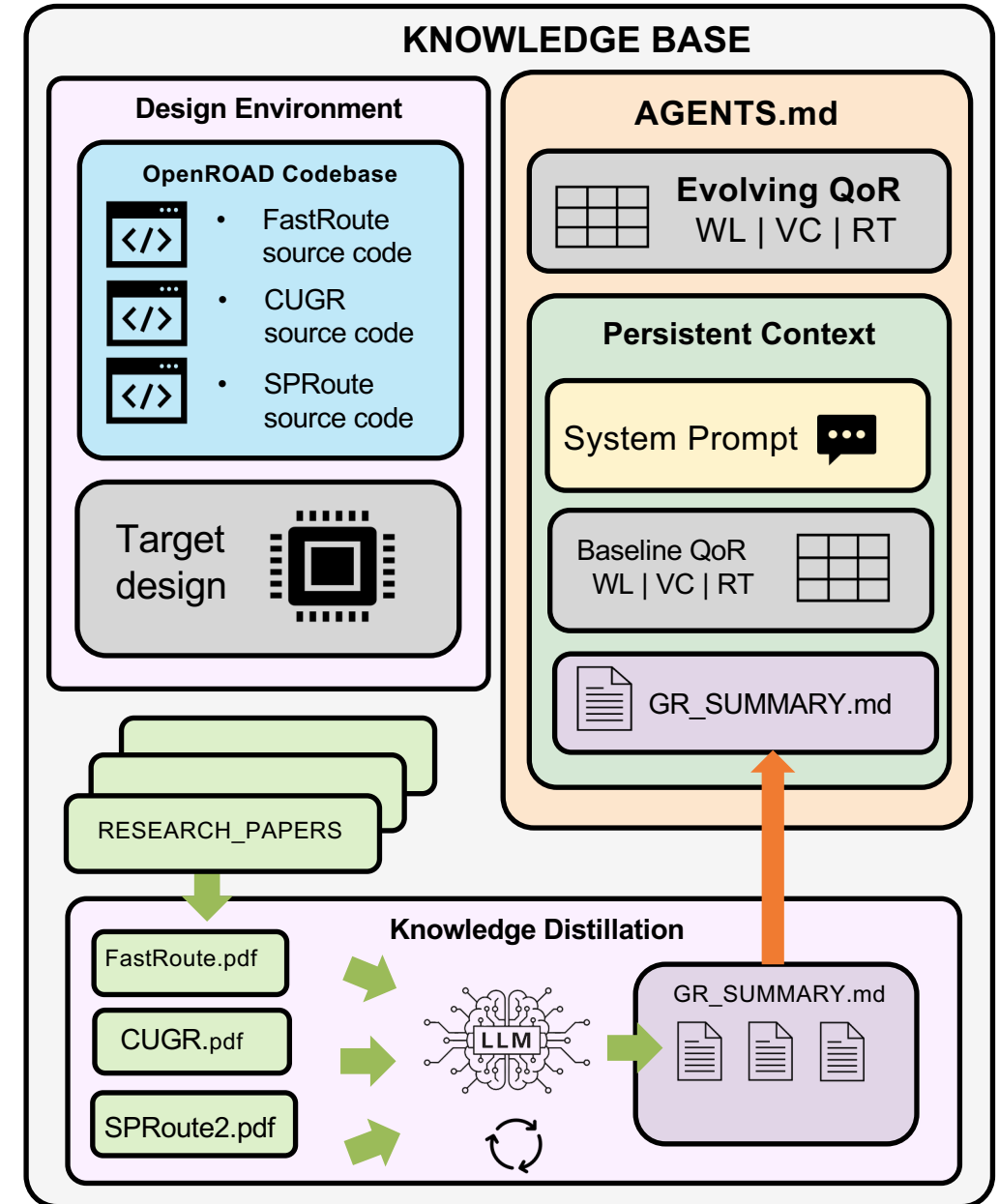
Vidya A. Chhabria
Arizona State University
Tempe, Arizona, US
vachhabr@asu.edu

Core idea – (GR-Evolve)

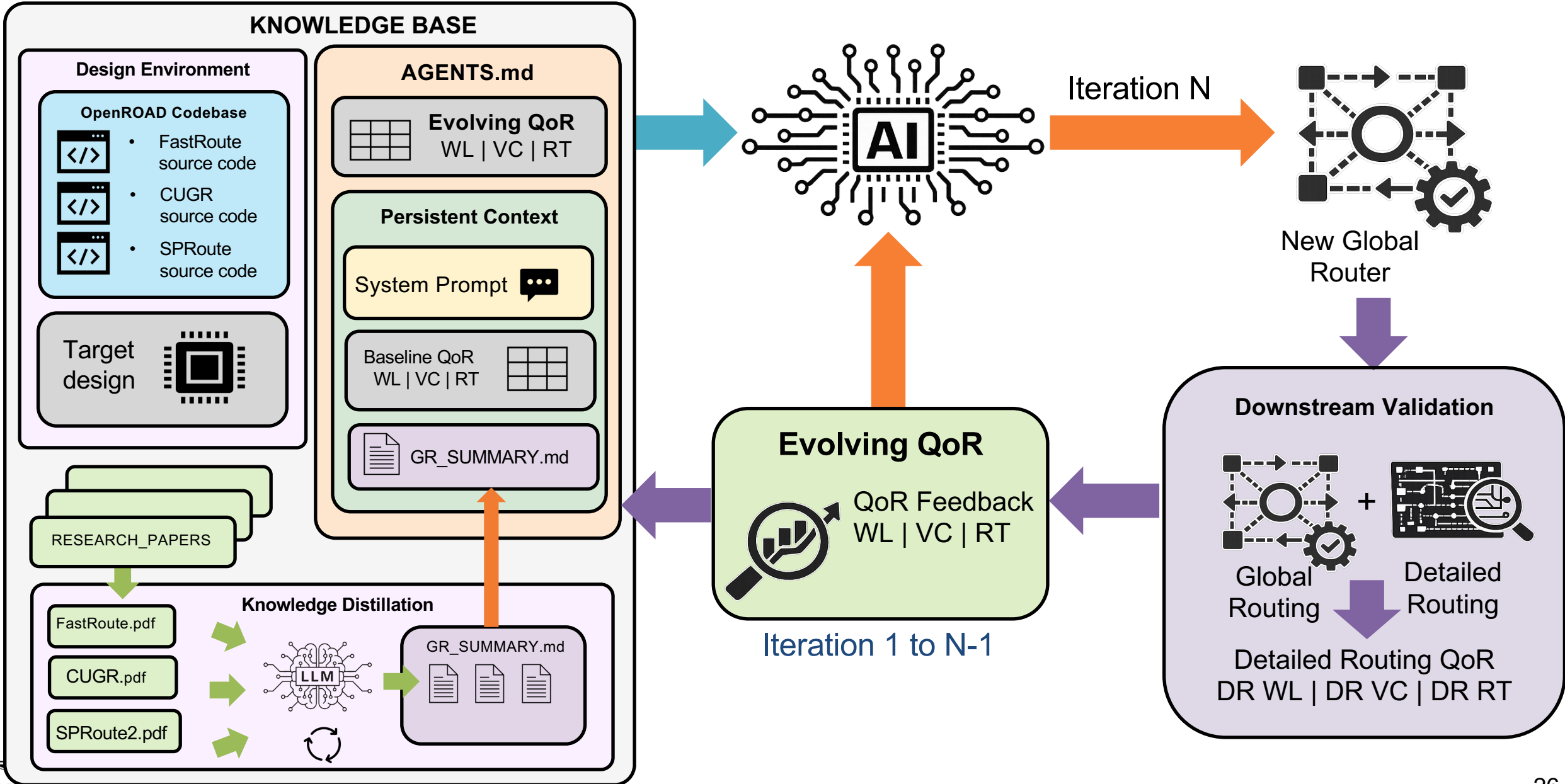


GR-Evolve's knowledge base

- **Design environment comprises**
 - OpenROAD codebase: FastRoute, CUGR, SPRoute
 - ORFS environment (design files, PDF libraries, etc.)
- **Knowledge distillation**
 - Research papers
 - LLM generated summaries
- **Structured context**
 - System prompt
 - Baseline QoR
 - Summary files from knowledge distillation

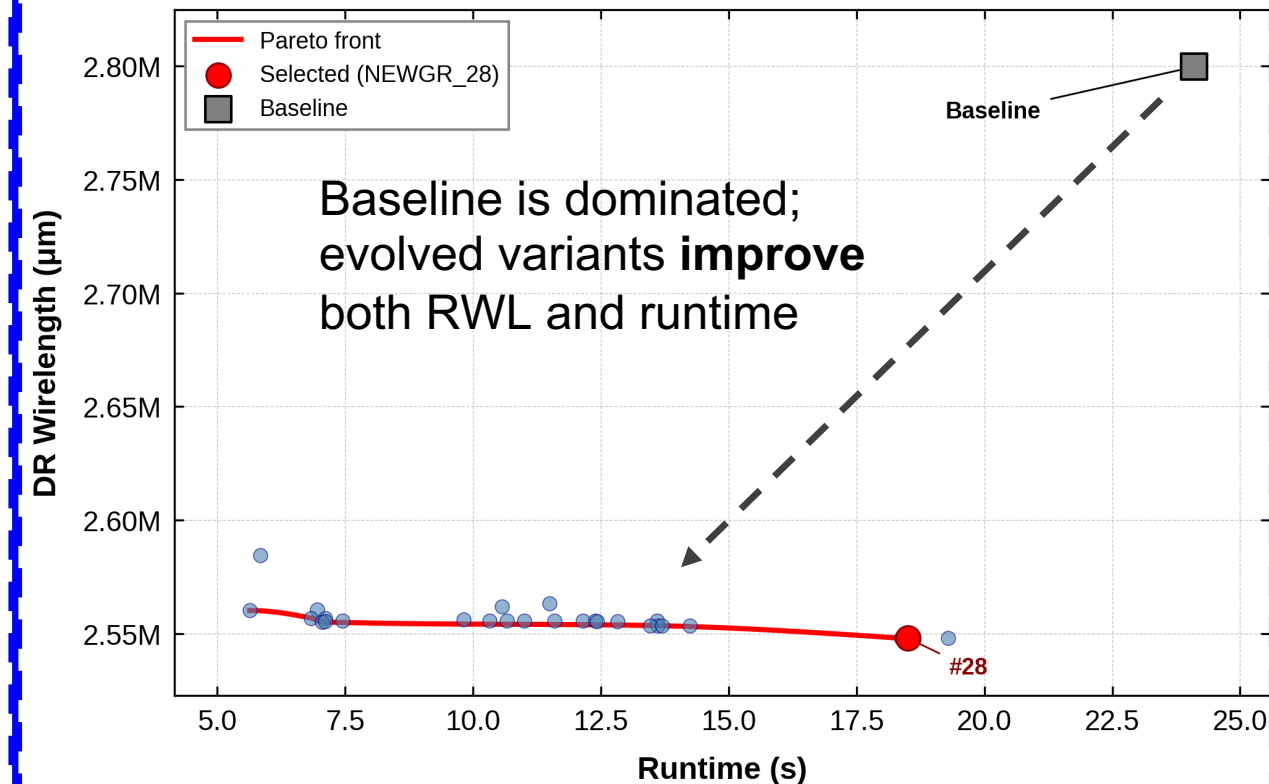


Context-driven code evolution

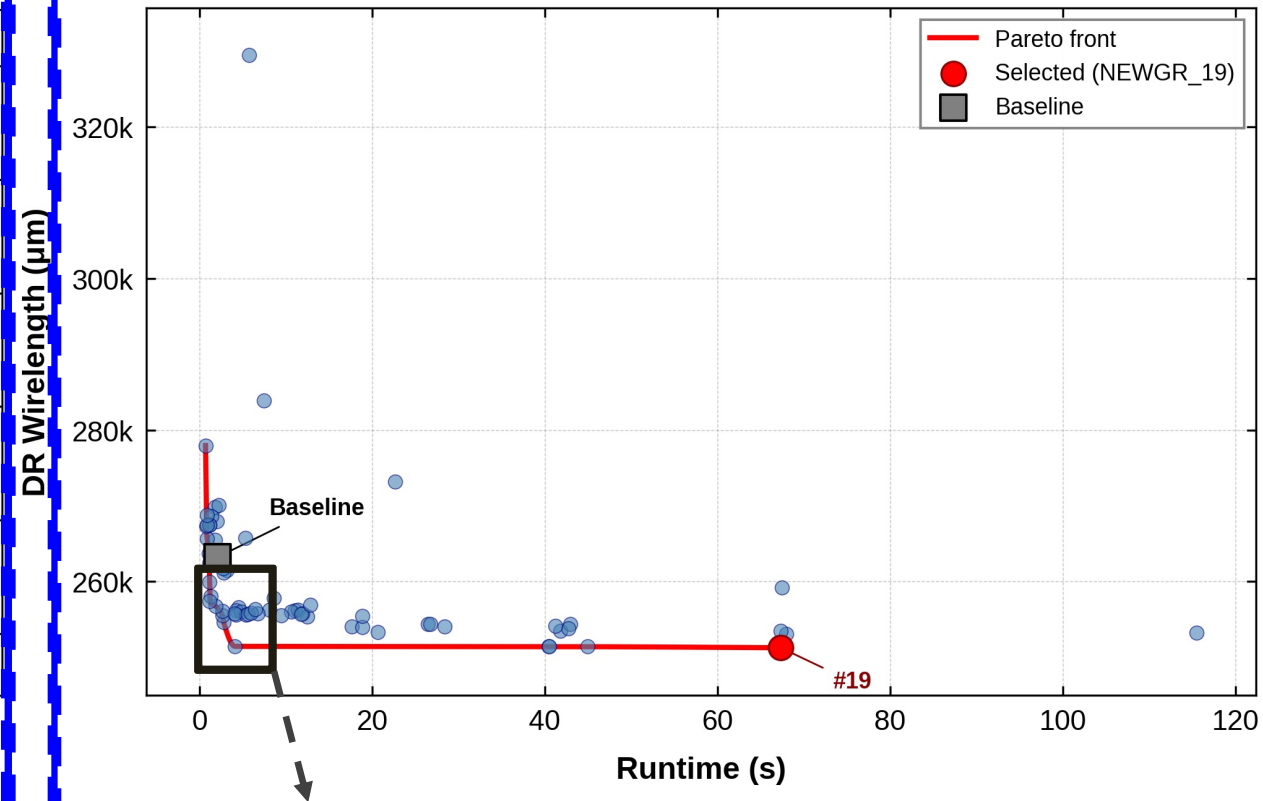


Design-tool co-exploration

CUGR_SWERV — NanGate45
DR Wirelength vs. Runtime



CUGR_AES — NanGate45
DR Wirelength vs. Runtime



Design-tool co-exploration: the router is no longer fixed; it adapts to the design

What does the agent actually change?

```
110 void CUGR::mazeRoute(std::vector<int>& netIndices){  
111     GridGraphView<CostT> wireCostView;  
112     grid_graph->extractWireCostView(wireCostView);  
113     sortNetIndices(netIndices);  
114     SparseGrid grid(10, 10, 0, 0);  
115     for (const int netIndex : netIndices) {  
116         GRNet* net = gr_nets_[netIndex].get();
```

Before: one static grid for routing

After: adaptive candidate grid generation

```
235 void CUGR::mazeRoute(std::vector<int>& netIndices){  
236     GridGraphView<CostT> wireCostView;  
237     grid_graph->extractWireCostView(wireCostView);  
238     sortNetIndices(netIndices);  
239     for (const int netIndex : netIndices) {  
240         GRNet* net = gr_nets_[netIndex].get();  
241         const int hp = std::max(net->getBoundingBox().hp(), 1);  
242         const int maxShrink = std::max(constants_.maze_base_interval  
243                                     - constants_.maze_min_interval,  
244                                     0);  
245         const int adaptiveInterval = std::max(  
246             constants_.maze_min_interval,  
247             constants_.maze_base_interval - std::min(maxShrink, hp / 3)  
248         );  
249         const int denseInterval  
250             = std::max(constants_.maze_min_interval, adaptiveInterval);  
251         const int coarseInterval = adaptiveInterval + 2;  
252         std::vector<CandidateGrid> candidateGrids;  
253         pushGridCandidate(  
254             candidateGrids, adaptiveInterval, netIndex + hp, netIndex  
255         );  
256         pushGridCandidate(  
257             candidateGrids, denseInterval, netIndex * 3 + hp, netIndex  
258         );  
259         pushGridCandidate(candidateGrids,  
260             constants_.maze_min_interval,  
261             netIndex * 11 + hp * 5,  
262             netIndex * 13 + hp);  
263         pushGridCandidate(  
264             candidateGrids, coarseInterval, netIndex * 5 + hp * 3, netIndex
```


Post-global routing metrics

Design	Evolved router	Wirelength % improvement	Via count % improvement	Runtime (s)	Baseline runtime (s)	Lines of code change
AES	FR_AES	0.13%	0.03%	2.973	6.62	561
	CUGR_AES	6.05%	6.06%	63.087	2.03	1329
	SPR_AES	7.72%	0.27%	0.833	1.41	1977
IBEX	FR_IBEX	Runtime degradation		80.342	2.46	745
	CUGR_IBEX			2.208	2.14	36
	SPR_IBEX			8.894	1.33	953
JPEG	FR_JPEG	4.76%	58.78%	68.638	6.31	558
	CUGR_JPEG	8.61%	5.72%	4.836	4.7	2537
	SPR_JPEG	6.07%	-21.65%	36.931	4.21	790
SWERV	FR_SWERV	-2.68%	-14.28%	40077.26	54.30	451
	CUGR_SWERV	10.81%	15.79%	28.69	24.10	4037
	SPR_SWERV	WL gains with similar runtime		5.78	9.15	1054
ARIANE136	FR_AR136			766.17	182.36	451
	CUGR_AR136			34.86	68.12	1257
	SPR_AR136	0.00%	0.00%	17.04	20.16	2047

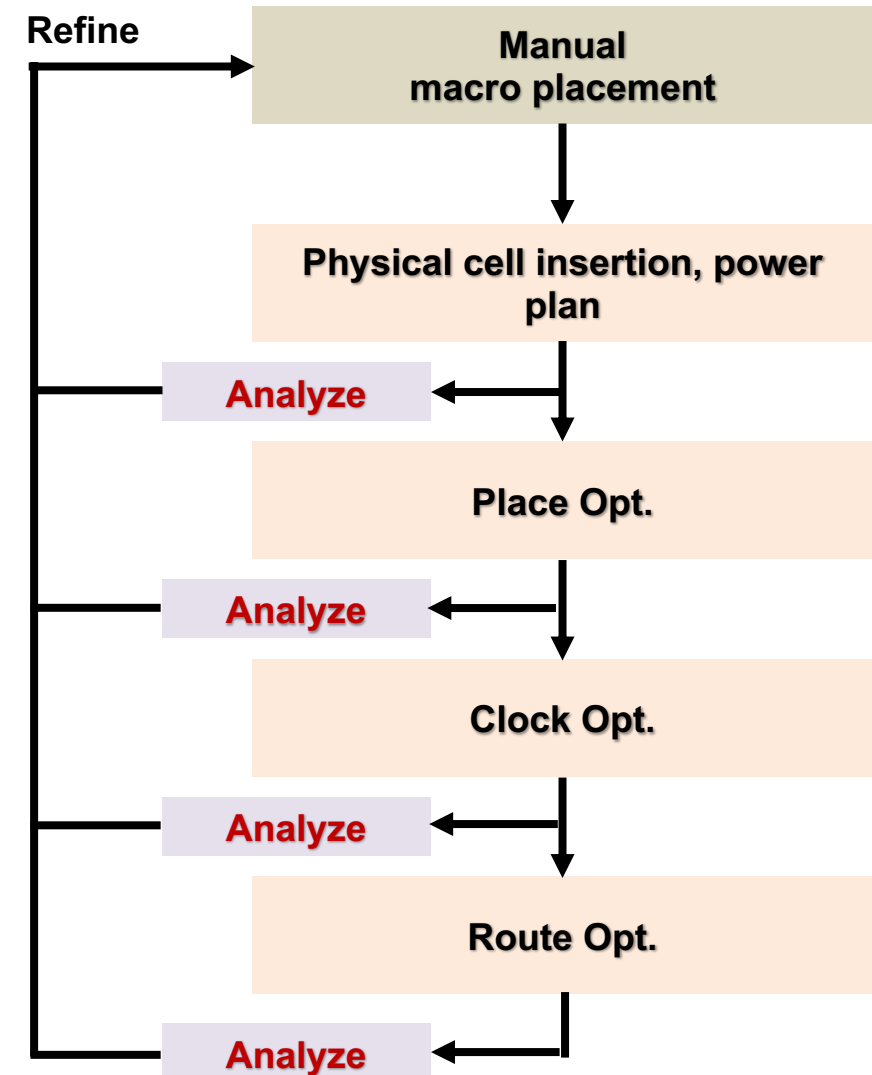
Be careful of what you asked for

Evolving Macro Placement

**MAGE: Human-Like Macro Placement via Agentic
Multimodal Reasoning**

How human engineers do macro placement?

- **Assimilate ideas from:**
 - Design architecture details from front-end, whitepapers, etc.
 - Commercial macro placement solutions (tools)
- **Extract similar groups (i.e., tiles, cores, etc.)**
 - Generate macro placement for each reference group
 - Transform reference macro placement for the final solution
- **Follow a list of principles**
 - Macro stacking, grouping, channels, symmetry, IO keep out, notch and pocket avoidance, macro pin accessibility
- **Based on the above try out multiple solutions**
 - Iterate and converge one macro placement solution
- **In industry, macro placement is still predominantly manual and expert-driven**
- **Goal:** Leverage visual language models to **encode the expert human engineer's spatial intuition** and automate the iterative, expert-driven macro placement flow



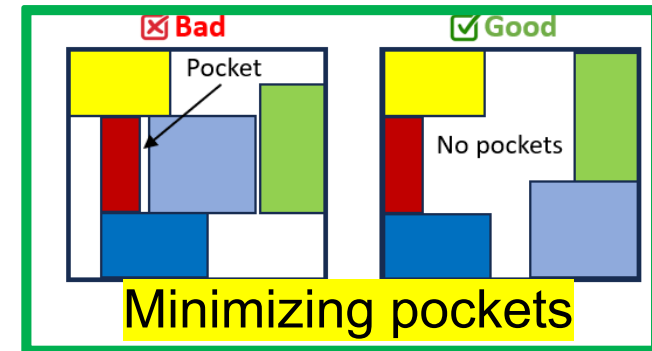
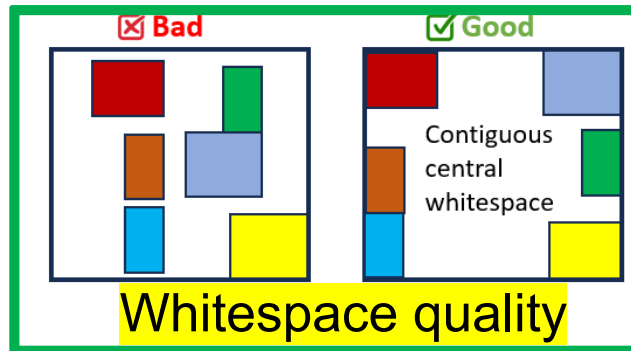
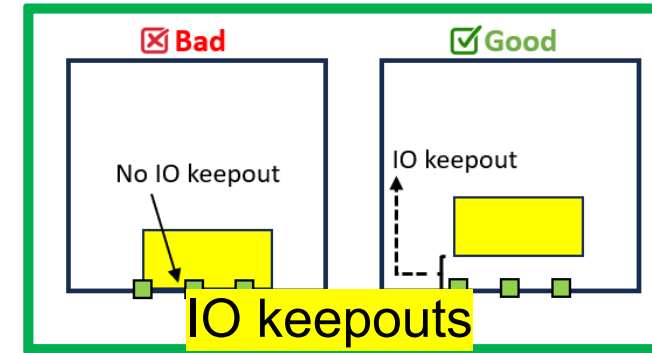
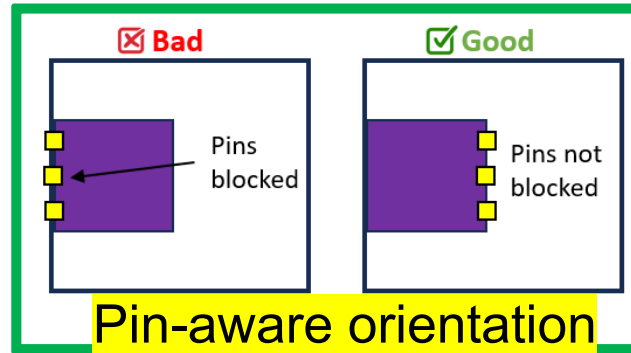
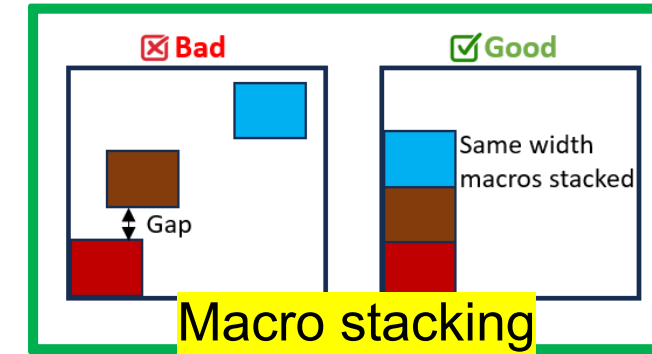
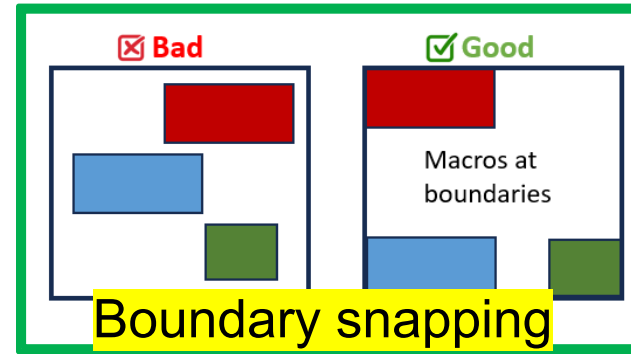
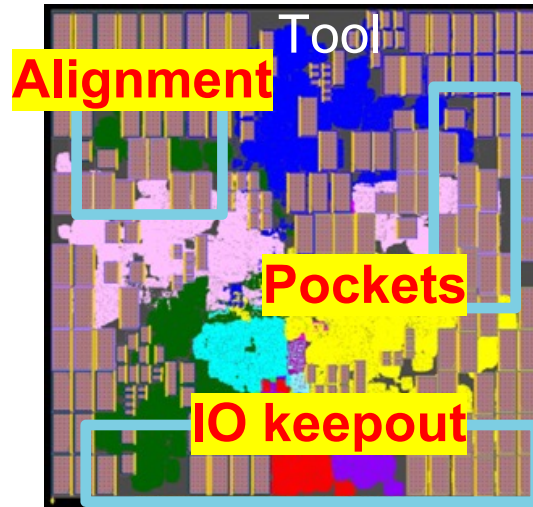
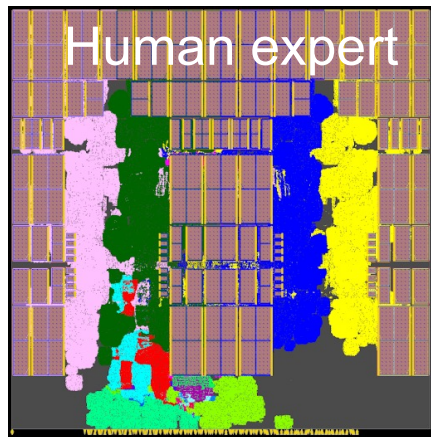
Problem statement – generate human-like macro placement

- **Given**

- Hard macros $M = \{m_1, \dots, m_n\}$ with fixed dimension (w_i, h_i)
- Core region C (possibly rectilinear), IO pin location, netlist N
- Reference placement $X^0 = \{(x_i^0, y_i^0, o_i^0)\}$

- **Constraints**

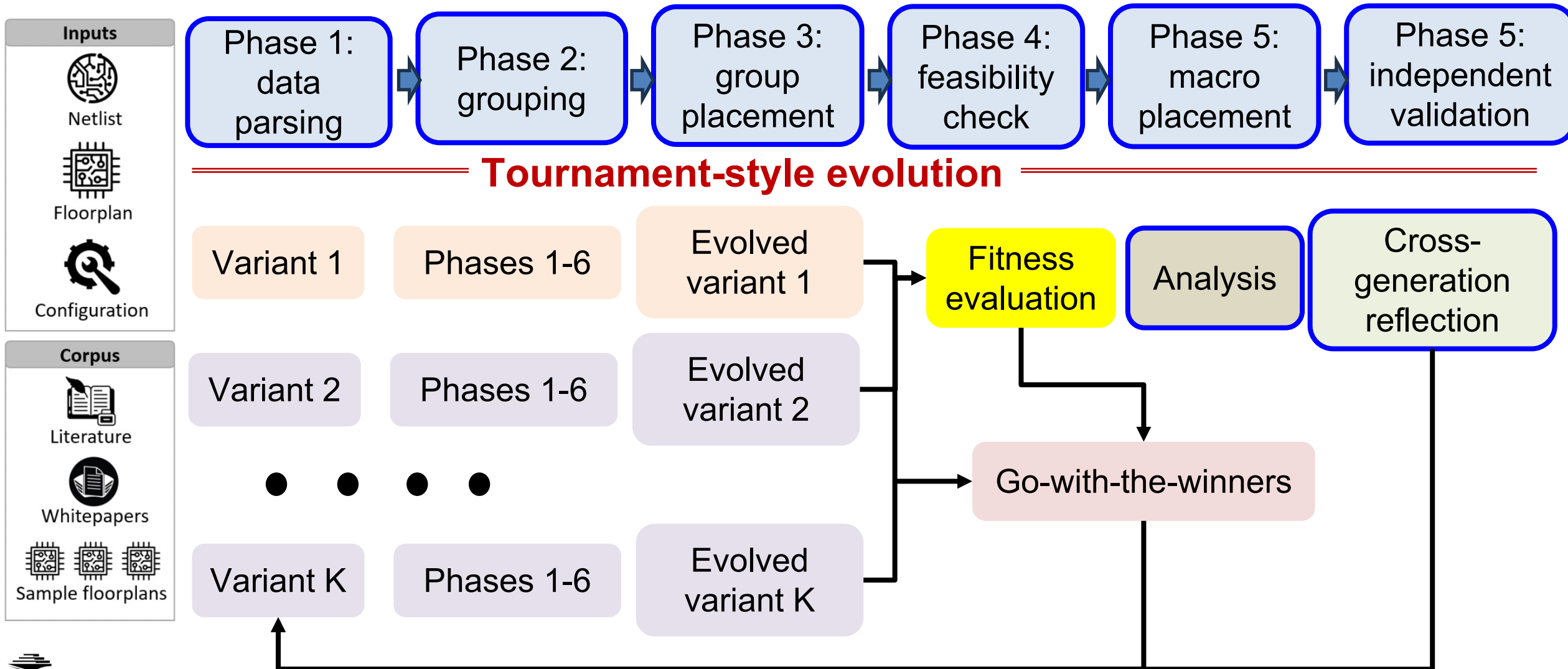
- All macros must reside within core boundary; no overlap between macros
- Must follow human expert rules (i.e., stacking, symmetry, ...)



Black Parrot Quad Core (BP)

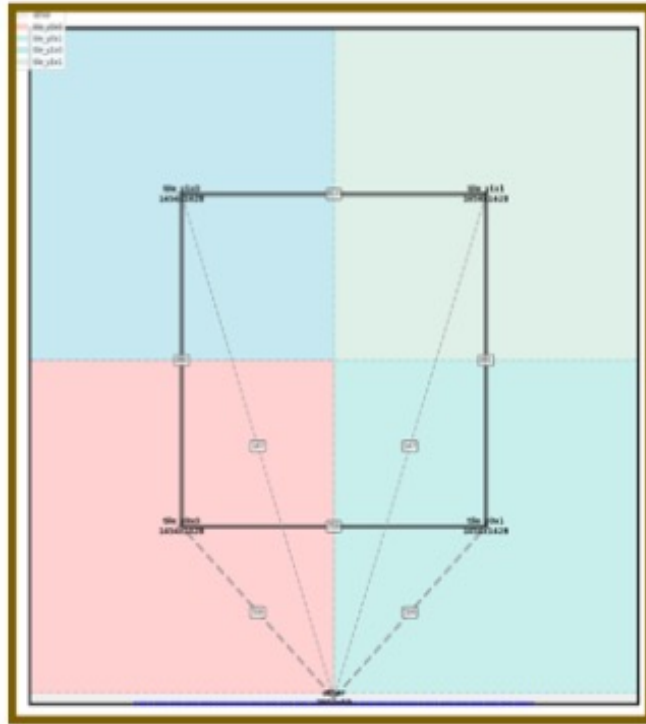
MAGE – Macro Placement Agentic Engine

- Core idea:** leverage vision + encode human placement rules in natural language prompts (boxes with blue borders indicate agent-driven actions)

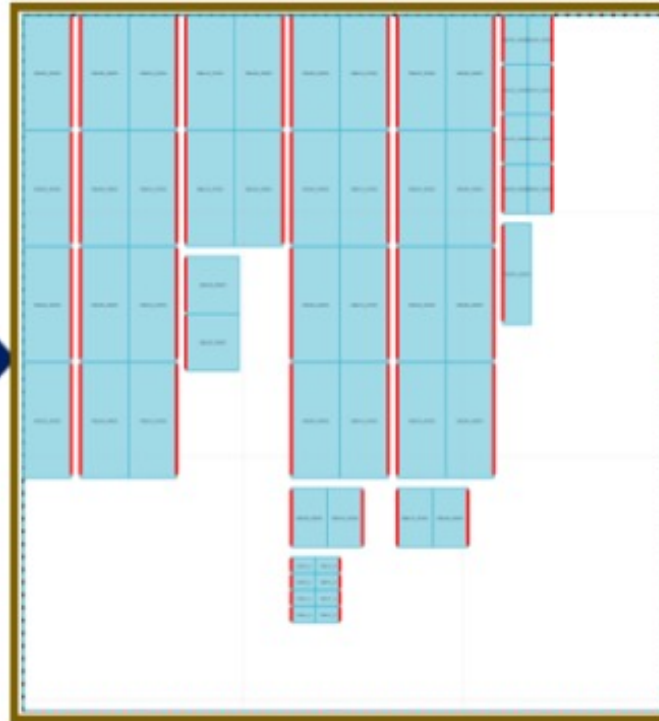


MAGE's macro placement recipe (vision everywhere)

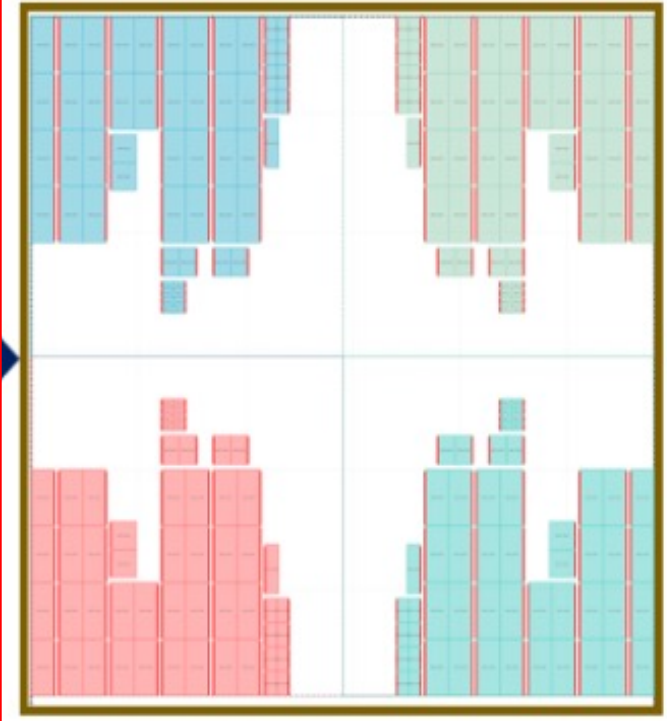
- **Step 1: find macro groups and tile the core for each group**
- **Step 2: generate macro placement for reference group**
- **Step 3: transform reference macro placement to other groups and merge**



After phase 3 (group placement)



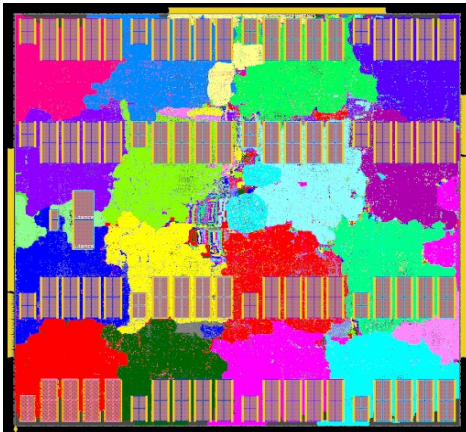
After phase 5 – reference placement



After phase 5 – merge and transform

MAGE's knowledge corpus

- **Design architecture documents**
 - Manuscripts describing architecture, dataflow, and memory hierarchy for designs
 - Mimics the architectural guidelines backend engineers receive from frontend
- **Macro placement domain knowledge**
 - Macro placement papers from related work, class notes, etc.
 - Encodes macro placement experience – mostly embedded in prompts
- **How are these provided and used?**
 - JSON summaries, PDF docs
 - Parser agent looks for design specific summaries and rules



MemPool Group

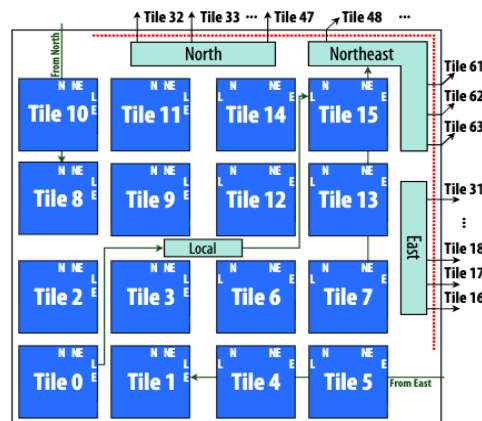


Figure 3a

```
## Tile Layout
...
Row 3: T4  T5  T6  T7
Row 2: T0  T3  T14 T15
Row 1: T1  T2  T11 T13
Row 0: T8  T9  T10 T12
...
```

Reasoning without
corpus

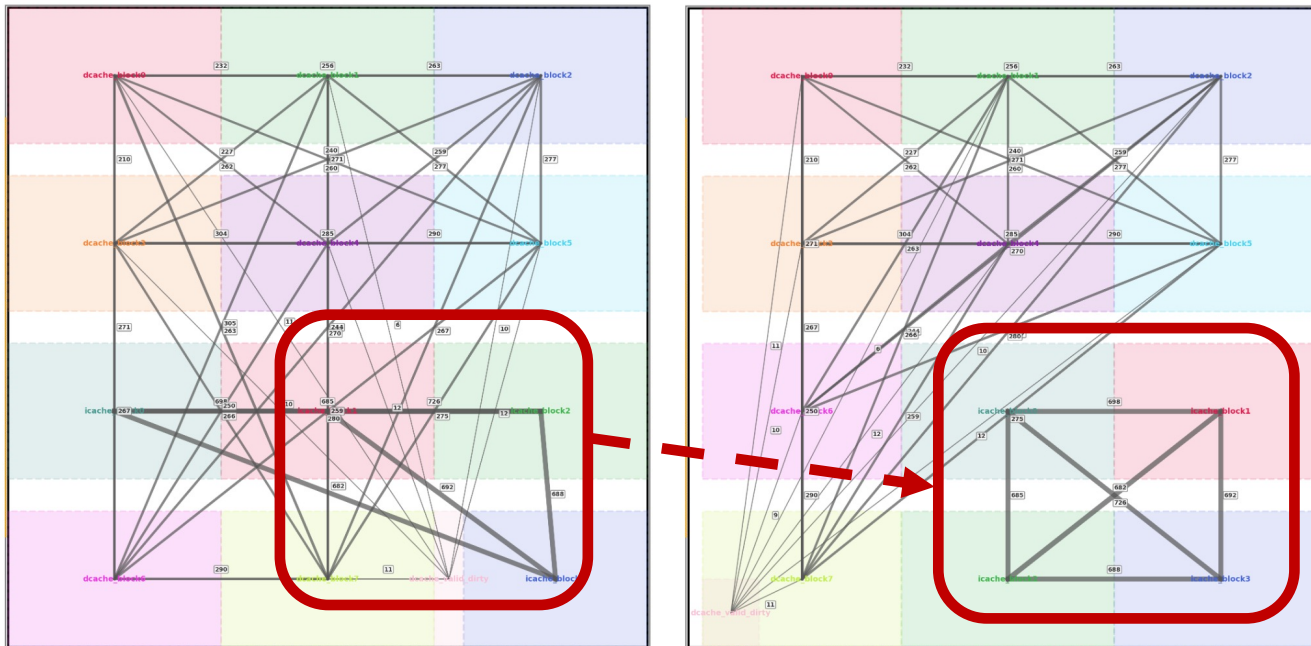
```
### Placement Reasoning
#### Tile Layout (Following MemPool Paper Figure 3a)
...
Row 3: T10 T11 T14 T15 (near top edge - TCDM IOs)
Row 2: T8  T9  T12 T13
Row 1: T2  T3  T6  T7
Row 0: T0  T1  T4  T5 (near bottom - no IOs)
...
```

Reasoning with
corpus

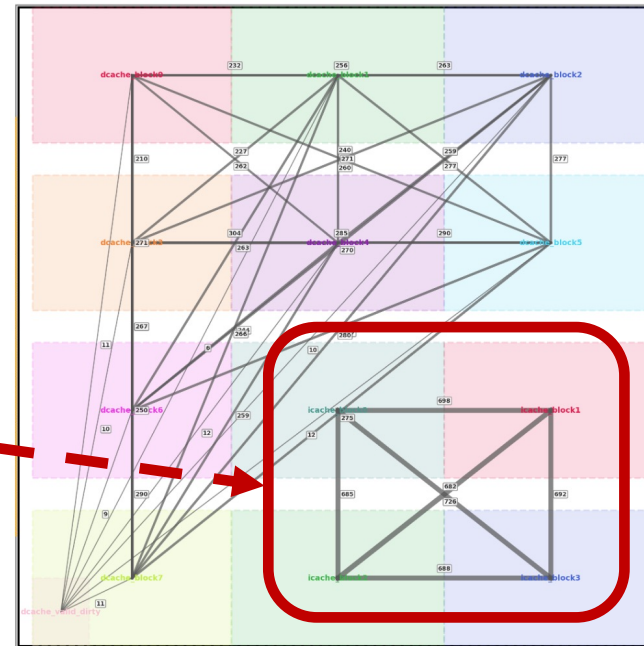
How is vision leveraged? – an example

Hierarchy-aware grouping (excerpt)

You are an expert who finds similar modules (logical hierarchy) in the design so that the macro placement task can be broken down into sub-tasks. Calculate group connections - include ALL connectivity, not just macro-to-macro. *If your connection matrix is sparse, you are likely counting only direct macro-to-macro nets.* Identify similar groups: Groups with the same macro count and same macro types/dimensions. These will share mirrored placement solutions.



Before



After



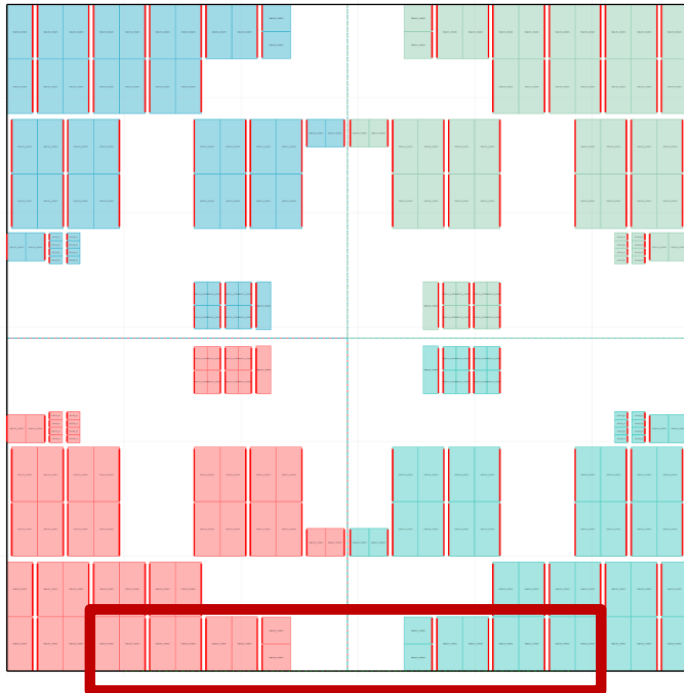
Visual feedback

- **Thick flylines** between icache blocks (682-726 nets) - **these are the strongest connections**
- icache_block0,1,2 in row 2 have **short flylines** between them (good)
- [] **Move** icache_block3 adjacent to icache 0,1,2 (row 2/adjacent to row 2)
- [] Consider **pulling** left-column groups slightly right to create IO routing space
- [] Consider making dcache_valid_dirty **smaller** or **merging** it into an adjacent dcache group

Enforcing human-likeness through vision

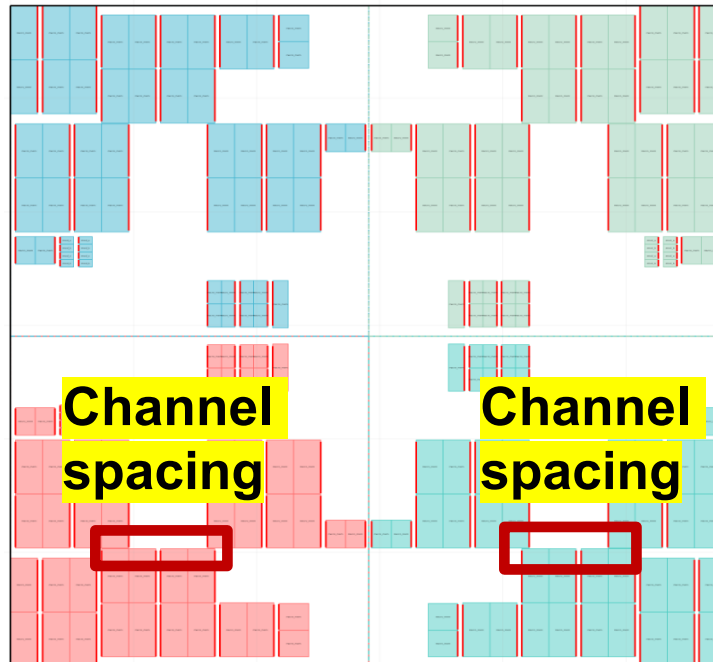
- **Agent runs validation scripts + sees the macro placement**
- Analyze the macro placement → generate report → suggestions how to fix violations (if any) ← **maintain history to prevent oscillations**

Iteration 0

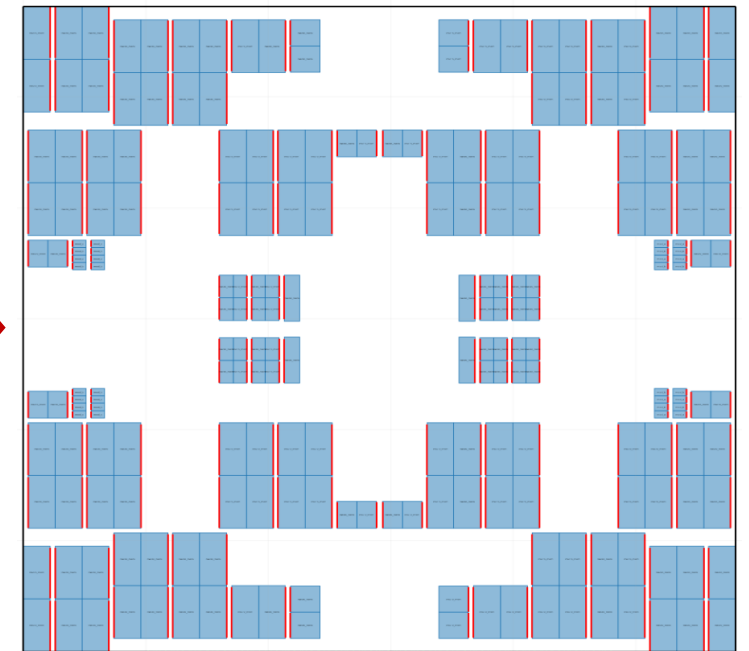


IO keep out violation

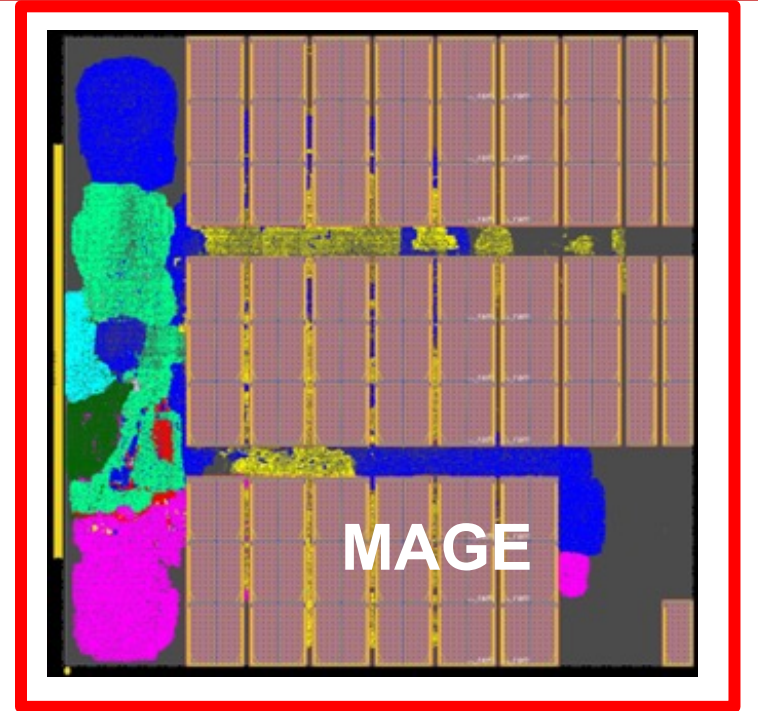
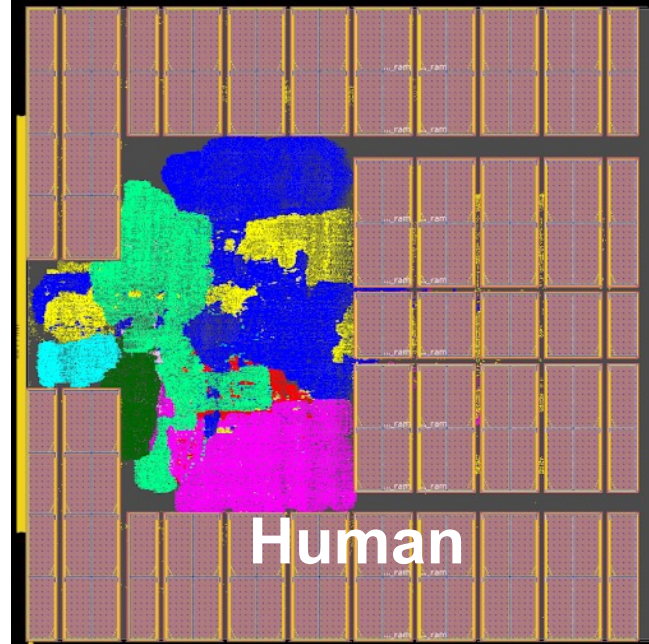
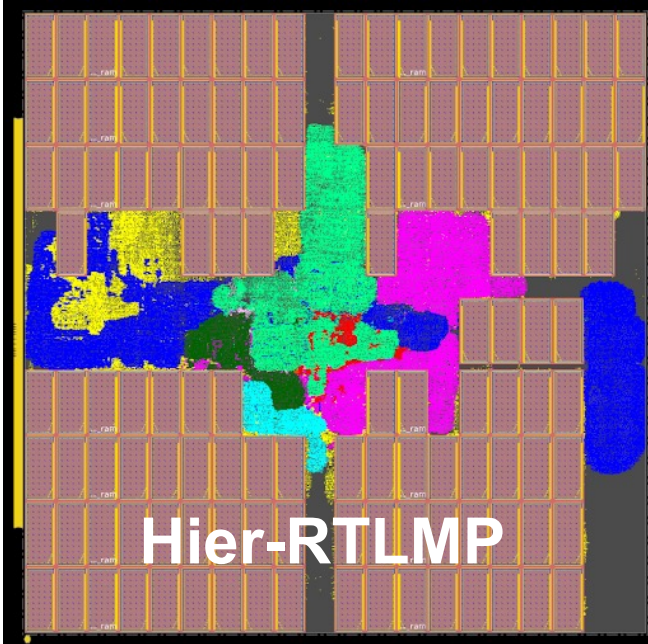
Iteration 1



Iteration 2

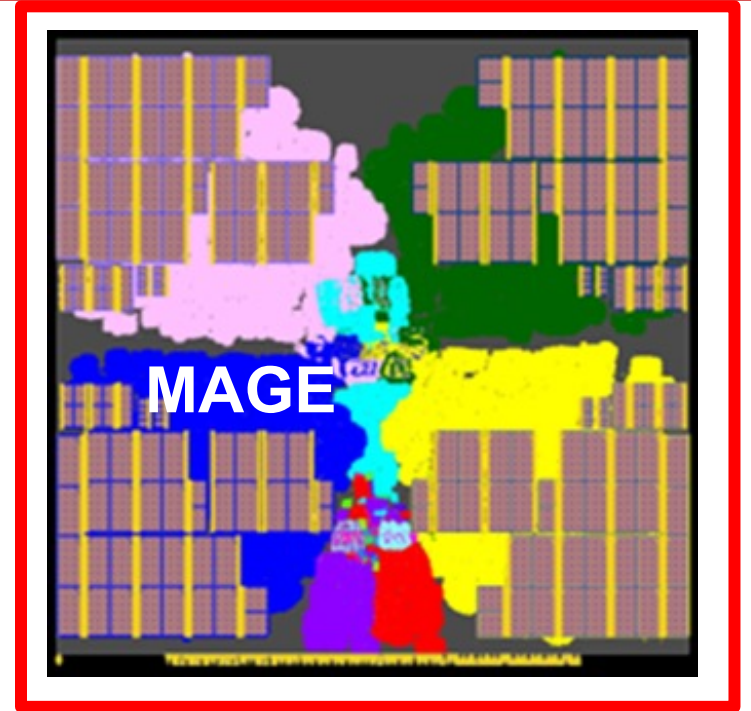
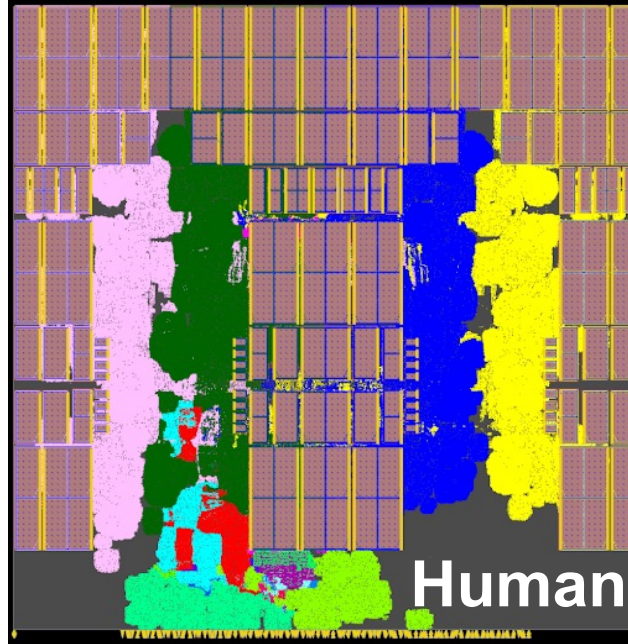
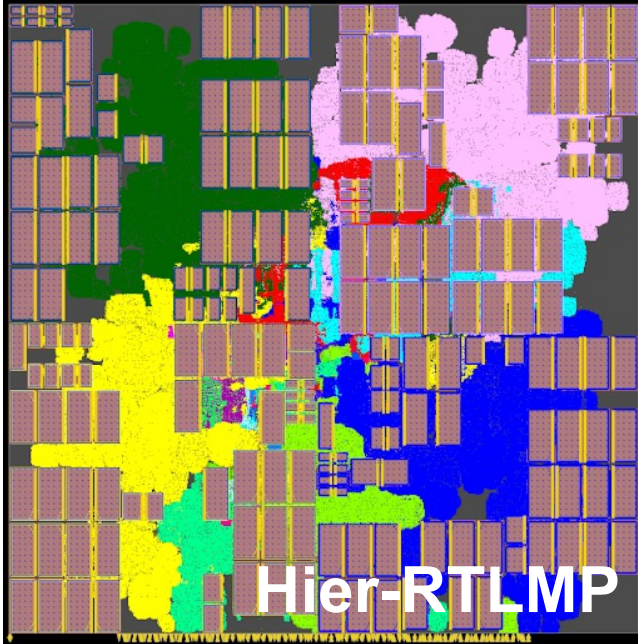


Comparisons with other macro placers (Ariane, NG45)



Macro placer	RWL (mm)	Power (mW)	WNS (ps)	TNS (ns)
Hier-RTLMP	5297	836	-165	-223.4
Human	4681	832	-88	-46.8
MAGE	4424	832	-81	-32.7

Comparisons with other macro placers (BP, NG45)



Macro placer	RWL (mm)	Power (mW)	WNS (ps)	TNS (ns)
Hier-RTLMP	28866	4512	-150	-456.6
Human	25916	4470	-97	-321.9
MAGE	26987	4479	-88	-139.7

Evolving an Entire Tool (ABC)

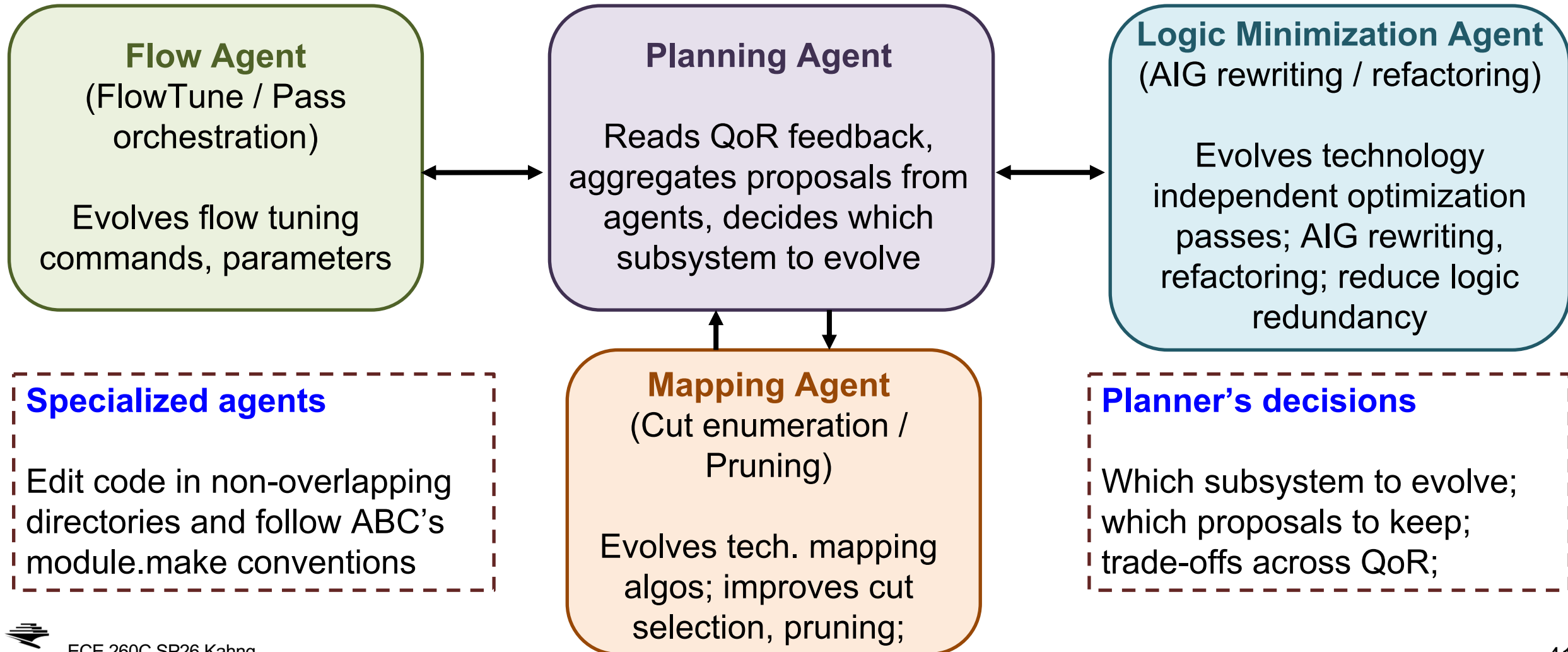
Autonomous Evolution of EDA Tools: Multi-Agent Self-Evolved ABC

Cunxi Yu
NVIDIA Research
University of Maryland
USA

Haoxing Ren
NVIDIA Research
USA

ABC-Evolve: can EDA tools evolve themselves?

- Decompose ABC into subsystems – **let specialized agents evolve their own code**



Correctness guardrails in ABC-Evolve

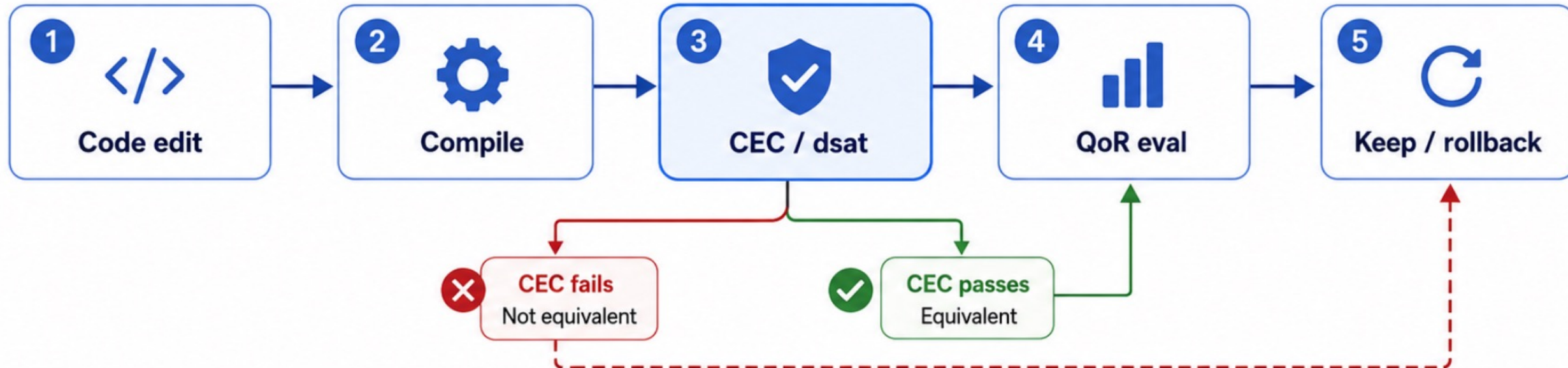
- **Why guardrails matter?**

- Broken synthesis can appear to improve QoR
- Compilation alone does not guarantee correctness

- **What ABC-Evolve does?**

- Equivalence checking filters invalid edits before benchmark eval.
- After every code modification, ABC is compiled
- CEC is run before any QoR measurement
- Only correctness-preserving edits proceed to evaluation

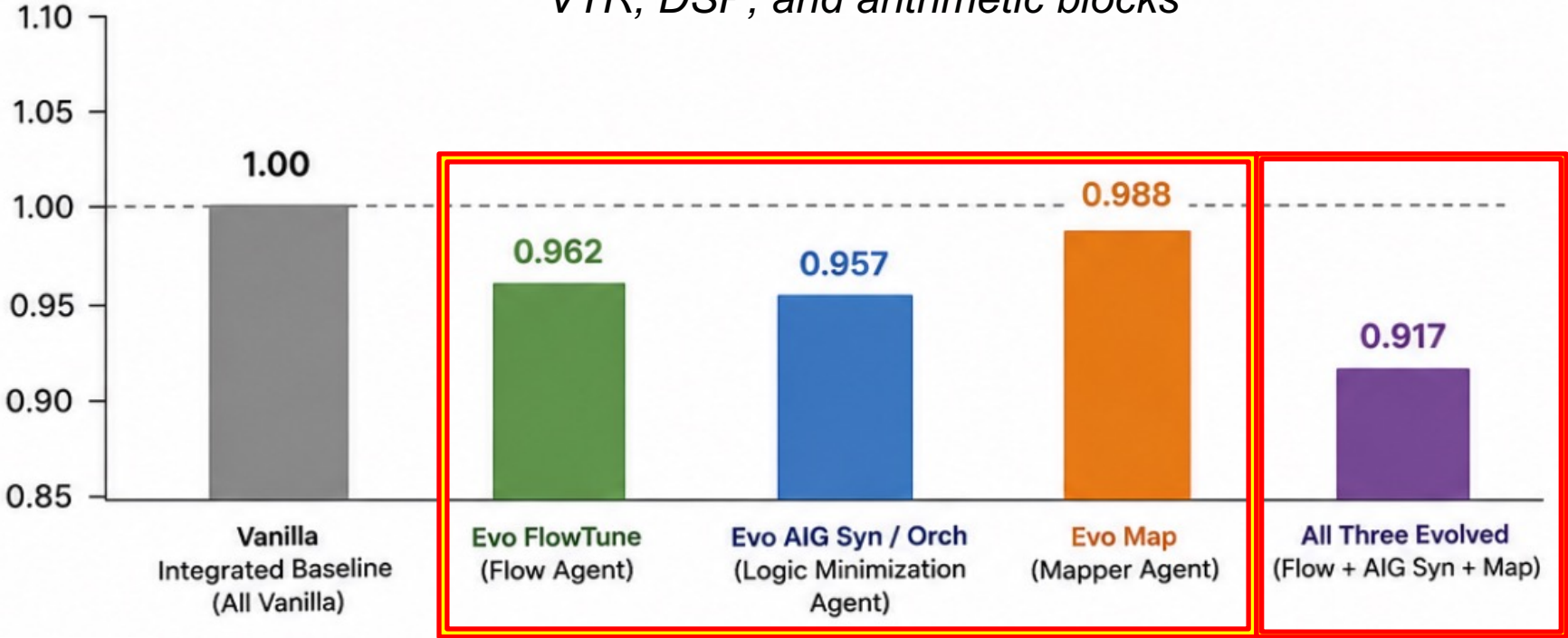
Autonomy become economically viable only when invalid edits are rejected early (!)



Results: co-evolution compounds

Normalized QoR
(Area-Delay Product)
Lower is Better

*Evaluated across ISCAS'85/'89/'99, EPFL,
VTR, DSP, and arithmetic blocks*



Normalized QoR (Area-Delay Product)	
Vanilla Integrated Baseline	1.00
Evo FlowTune	0.962
Evo AIG Syn / Orch	0.957
Evo Map	0.988
All Three Evolved	0.917

**Best QoR Improvement
with Full Co-evolution**
~8.3%
vs. Vanilla Integrated Baseline

**Takeaway**
**Co-evolution Beats
Isolated Evolution**

- ✓ Single-agent evolution delivers modest gains (↓ 1–4%).
- ✓ Pairwise co-evolution provides stronger improvements (↓ 6–8%).
- ✓ Joint evolution of all subsystems delivers the best result (↓ ~8.3%).

**Single-agent gains are modest; multi-subsystem co-evolution gives the best QoR.**

Lessons Learned: Toward an Autonomous EDA R&D SWE

What four systems (**HELIX, (GR + ABC)-Evolve, MAGE**) agree on

Grounding + physical feedback

- Domain grounding and physical feedback are complementary; not substitutable
- The agent must know the domain and see the physical consequence of its edits

Correctness guards

- Correctness guards are what make autonomous evolution economically viable
- Without automatic gates, you waste resources on invalid states and false QoR

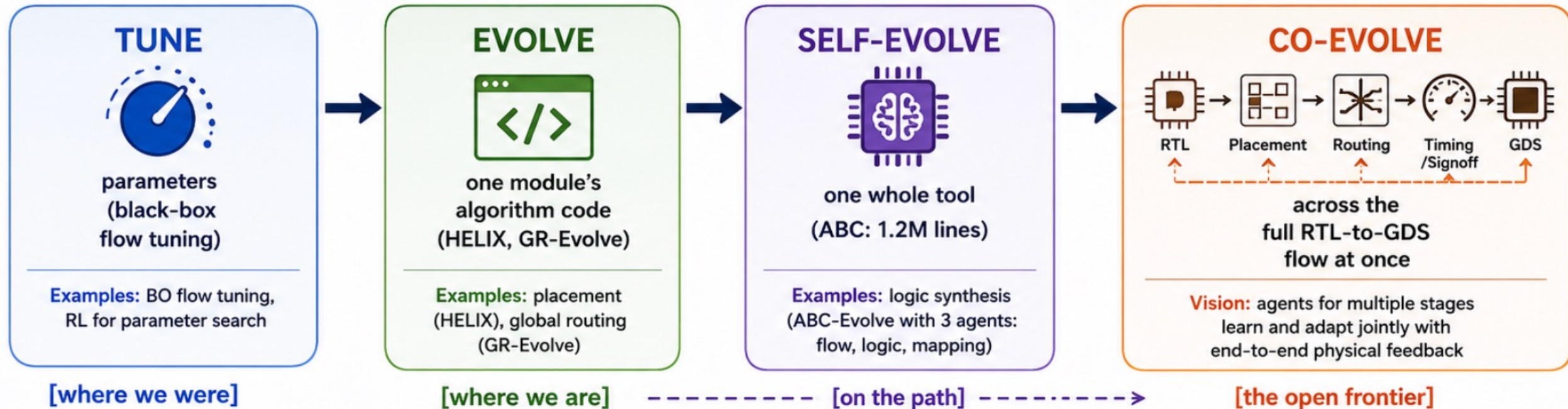
The loop matters

- The evolutionary loop, not raw model capability, drives the result
- Better models help, but the scaffolding around it does the heavy lifting

Agents amplify priors

- Today's agents amplify human priors – they do not yet invent paradigms
- Win is a force-multiplier on human-curated knowledge, not autonomous invention

The spectrum: where we are heading



The big picture: *Agentic PD systems are scaling in scope. The frontier is full-flow, multi-objective, and jointly optimized. We're not there yet – but the path forward is clear.*

How would an autonomous OpenROAD agent work?

- **Step 1:** Documentation as **compression**
- **Step 2:** Fuse documentation with **non-code context** → implementation plans
- **Step 3:** Implementation plans → **diff** → tests + compile checks + in-loop optimization for metrics
- **Step 4:** Diff → **PR**, with unit tests, comments, mergeability
- **Step 5:** In-house **PR auditor** reviews outputs of Step 4, optionally return to **Step 3**
- **Building the autonomous loop:**
 - Feasible today atop **OSS agents** (e.g., Cline, Claude Code, Codex)
 - Harder to integrate with **closed-source agents** (e.g., Cursor)

The big shift in EDA ...



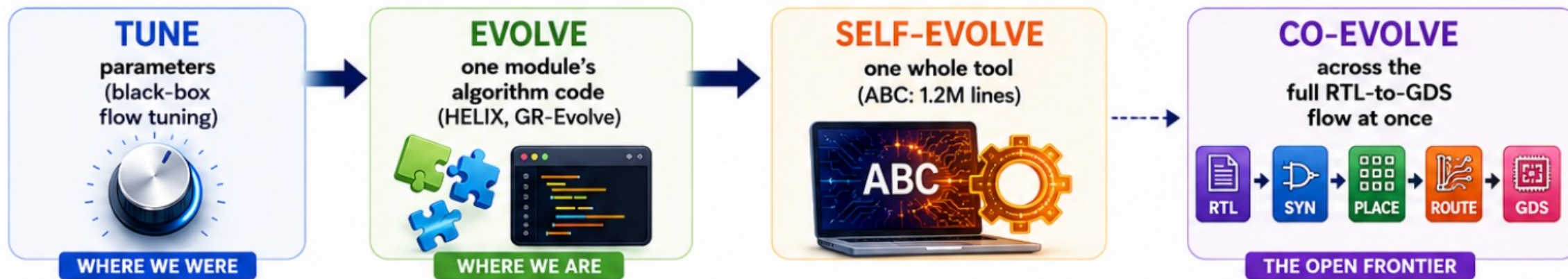
The unit of optimization in EDA moved up a level — from parameters to **the algorithm's source code** itself.

And the agent's real deliverable isn't a black-box gain — it's an **interpretable** change a human engineer can read, understand, and adopt.



The placer just got its **first new idea in 20 years** — and it came as a **diff** a person can read.

THE SPECTRUM (ONE MORE TIME)



These results exist because of decades of EDA foundations, not despite them — every system bootstraps from human-built tools, literature, and benchmarks. The agent amplifies that work; it doesn't replace it

Thank you!

<https://vlsicad.ucsd.edu/>

<https://github.com/ABKGroup>

Agents optimizing the CAD flow (ORFS-agent)



Code
Available



Code
Reviewed



Code
Reproducible

ORFS-agent: Tool-Using Agents for Chip Design Optimization

Amur Ghose, Andrew B. Kahng, Sayak Kundu, Zhiang Wang
{aghose,abk,sakundu,zhw033}@ucsd.edu
UC San Diego, La Jolla, California, USA

* **ORFS = OpenROAD-flow-scripts** is a Makefile-based “flow runner”
(*cf. OpenLane/LibreLane, SiliconCompiler, mflowgen, Hammer, ...*) for OpenROAD

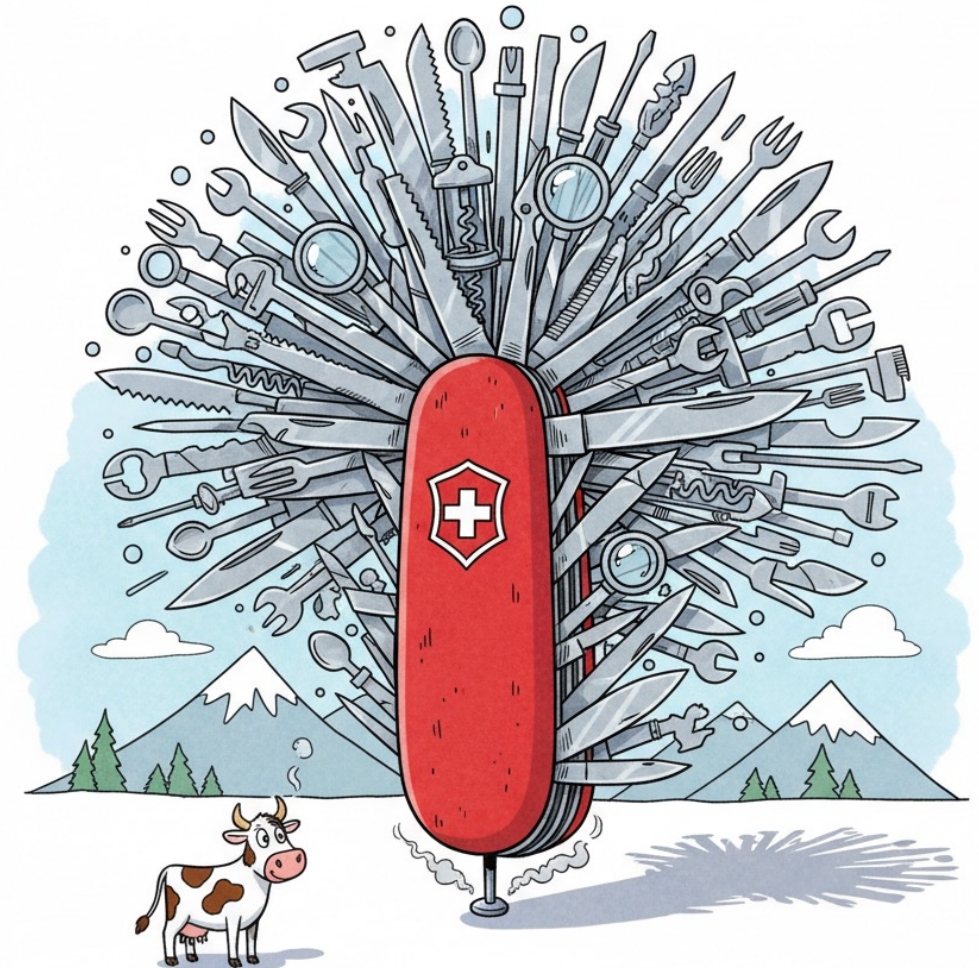
What is ORFS-agent

- LLM autotuner inside OpenROAD-flow-scripts
 - Parallel trials, metric-aware refinement
- Natural Language (NL) objectives
 - Single-/multi-objective (Wire Length, Effective Clock Period); with NL constraints
- Tool-using loop: INSPECT / MODEL / AGGLOMERATE
 - Optional Bayesian Optimization (BO) for sample-efficiency
- Model-agnostic
 - Benefits scale with foundation model strength

Design flow tuning is high-dimensional

- **1000s (actually, 10000s) of knobs in RTL-to-GDS**
- **Plain BO struggles**
weak domain priors; poor high-D scaling; objective design
- **Enter: Agents**
read logs; reason in context; call tools; iterate
- **Core loop**
propose → parallel ORFS runs → read metrics/logs → refine
- **Model-agnostic**
no fine-tuning; swap in stronger LLMs

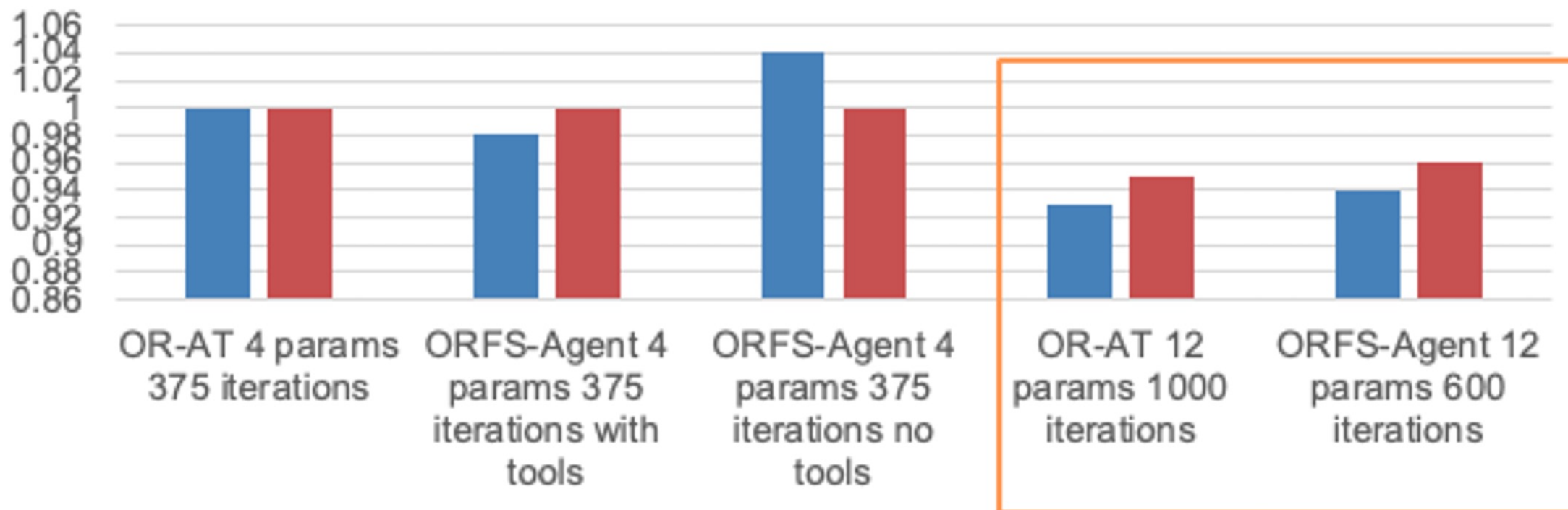
SWISS ARMY KNIFE



Semantics-aware search >> blind Bayesian Optimization

- **Classical BO treats the vector of *knobs* as opaque**
 - *Knobs* have **names** and **physics**
 - **Core Util** $\uparrow$ $\Rightarrow$ **congestion risk**
 - **Clock Period** $\downarrow$ $\Rightarrow$ **timing pressure**
- **Agent can exploit semantics and logs**
 - Recognizes regimes, uses priors (ranges, monotonicities), proposes meaningful batches
- **Loop**
 - **Read** logs $\rightarrow$ **summarize** context $\rightarrow$ **choose** sampling scheme (local sweep, bracket, exploratory LHS) $\rightarrow$ **refine**
- **Outcomes**
 - **Better** early signal
 - **Fewer** wasted trials than black-box BO at **similar** parallelism

Key outcomes



Comparison of ORFS-agent and OR-AutoTuner w.r.t. **wirelength** and **ECP**

Normalization: Results with OR-AT4 params and 375 iterations set as 1.0

- Baseline: OR-AT (4 vars, 375 iters) $\equiv 1.0$
- ORFS-agent can achieve $\approx 40\%$ fewer iters iso-QOR $\approx 13\%$ gains in WL or ECP (single-objective) [\(details in the paper\)](#)